PROJECT REVIEW NOTES / WEWITHU (APNACOACH)

Backend, Database & Core Features

Phase 4 Phase 5 Phase 6 fifteen tasks, real code references, flow diagrams

A review-ready reference for this codebase: how one request travels from React through Express, middleware and MongoDB, how the four data models are shaped, and how the six product features are wired from click to stored record.

STACK	React 19 Express 5 Mongoose 9 Groq Razorpay
SCOPE	Tasks 15 29 15 tasks 3 phases
INSIDE	6 flow diagrams findings register viva answers
BASIS	every claim cross-checked against repository source



WHAT IS INSIDE

PART I
Reading map
snapshot 6-step model

PART II
Request lifecycle
Tasks 15 19

PART III
Database & models
Tasks 20 23

PART IV
Core features
Tasks 24 29

PART V
Review pack
flows answers cheat sheet

HOW TO READ

- **Before the review** Read Parts II IV in order; every task ends with a sentence you can say out loud.
- **During the review** Keep Part V open: findings register, flow gallery and the one-page cheat sheet.
- **When asked for improvements** Quote a finding, its file, and the smallest correct fix that reads as maturity.

Read Parts II IV in order before the review; keep Part V open during it.

github.com/amitsharma4-5/WeWithU branch arena/50cee119-wewithu 38 pages

Contents

Fifteen tasks across three phases, plus a review pack.

Part I Reading Map and Project Snapshot 3

How to use these notes 03
1. Project snapshot 03
2. Repository map (where to look during the review) 03
3. The six-step mental model 04
4. Two details that impress reviewers 04
5. How the three phases connect 04
6. Legend used in these notes 04

Part II Backend Request Lifecycle (Phase 4) 5

The big picture 05
Task 15 Understand server bootstrap 05
Task 16 Understand route files 08
Task 17 Understand authentication middleware 10
Task 18 Trace the current-user API from frontend to database 12
Task 19 Understand errors and HTTP status codes 14

Part III Database and Data Models (Phase 5) 15

First, understand the architecture 15
Task 20 Understand the MongoDB connection 15
Task 21 Read each Mongoose schema 17
Task 22 Trace a create, read and update operation 19
Task 23 Understand data validation 21

Part IV Core Product Features (Phase 6) 23

The overall architecture 23
Task 24 Interview workflow 23
Task 25 AI question generation and evaluation 25
Task 26 Resume workflow 27
Task 27 Placement test workflow 29
Task 28 History and reports 31
Task 29 Guidance and preparation pages 32

Part V Review Pack 34

A. Consolidated findings register 34
B. End-to-end flows you should be able to draw 35
C. Answer keys for the practice questions 36
D. Rapid-fire viva questions with model answers 36
E. Two-day revision plan 37
F. Demo script for the review (10 minutes) 37
G. Glossary 37
H. One-page cheat sheet 38

WEWITHU REVIEW NOTES

Reading Map and Project Snapshot

How to use these notes, what the product is, and the six-step request model that every later task refers back to.

How to use these notes

These notes cover **Phase 4 (backend request lifecycle), Phase 5 (database and data models) and Phase 6 (core product features)** of the WeWithU / ApnaCoach project. They are written for a **project review or viva**, so every point from the three phases is kept, explained in simple language, and connected to the real files in this repository.

There are two ways to read them:

Before the review: read Part II, III and IV in order. Each task ends with a "Say this in review" line you can repeat almost word for word.

During the review: use Part V. It contains the full endpoint map, the consolidated findings register, answer keys for all practice questions and a glossary.

Every technical statement here was checked against the code in this repository, not assumed from the phase text. Where the phase text and the code differ, the difference is called out explicitly.

> **NOTE**
>
> Name of the project The phase material calls the project **ApnaCoach** and mentions `apnacoach.onrender.com`. The code in this repository is branded **WeWithU** (`wewithyou.onrender.com`, `wewithyou Backend API`). The architecture, files and endpoints are the same. These notes use the repository names, because that is what a reviewer will open on your screen.

1. Project snapshot

Item	What this project actually uses
Product	AI mock interviews (HR + technical), 5-module placement drive, resume parsing, ATS check, career guidance, roadmaps, credits and payments
Frontend	React 19, Vite 7, Tailwind CSS 4, Redux Toolkit, React Router 7, Axios, Framer Motion (`motion`), Recharts, jsPDF
Backend	Node.js, Express 5, Mongoose 9, cookie-parser, cors, multer, pdfjs-dist, jsonwebtoken, Razorpay SDK
Database	MongoDB (Atlas) via `MONGODB_URL`, four collections: users, interviews, payments, placementtests
AI provider	Groq Chat Completions API, default model `llama-3.3-70b-versatile`, wrapped in `server/services/groq.service.js`
Authentication	Firebase Google sign-in on the client, then a backend JWT stored in an **HttpOnly cookie named `token`**
Payments	Razorpay order + HMAC-SHA256 signature verification, credits added on success
Reports	Interview report queries and placement report queries; PDF export happens on the client with jsPDF

2. Repository map (where to look during the review)

```
client/src/
  App.jsx                route table + "who am I" request at startup
  pages/                 Home, Auth, InterviewPage, InterviewHistory, InterviewReport,
                         Pricing, Guidance, ResumeTips, CompanyPrep, Roadmap,
                         PlacementTestPage, PlacementResultsPage
  components/            Navbar, Footer, Step1SetUp, Step2Interview, Step3Report, Timer
  redux/                 store.js, userSlice.js, interviewSlice.js
  utils/                 serverUrl.js, firebase.js, jobRoles.js

server/
  index.js               bootstrap: dotenv, express, cors, parsers, routes, listen
  config/                connectDb.js (mongoose), token.js (JWT signing)
  middlewares/           isAuth.js (cookie JWT), multer.js (PDF upload)
  routes/                8 routers mounted under /api/*
  controllers/           business logic for auth, user, interview, payment,
                         resume, questions, evaluate, placement
  models/                user, interview, payment, placement
```

```
18  services/              groq.service.js (AI), razorpay.service.js, scorer.service.js
```

3. The six-step mental model

Every request in this project can be explained with the same six boxes. Memorise this picture; it is the backbone of all three phases.

1. **React frontend** — axios.get / axios.post with ServerUrl and withCredentials — `client/src`
2. **Express bootstrap** — CORS, express.json(), cookieParser(), route mounts — `server/index.js`
3. **Router** — method + path match middleware list controller — `server/routes/*.js`
4. **isAuth middleware** — cookie token jwt.verify req.userId next() — `middlewares/isAuth.js`
5. **Controller + model** — validate, askAi(), Mongoose query (findById / create / save) — `controllers` `models`
6. **Response to React** — status code + JSON Axios Redux or local state — `client/src`

1. **React frontend** a page or component calls Axios with `ServerUrl` + `"/api/..."`.
2. **Express server** `server/index.js` runs CORS, JSON parsing and cookie parsing, then forwards the request to a router.
3. **Router** `server/routes/*.js` matches HTTP method + path and lists the middleware and controller to run.
4. **Authentication middleware** `isAuth.js` reads the `token` cookie, verifies the JWT and sets `req.userId`.
5. **Controller and model** the controller validates input, calls `askAi()` and/or a Mongoose model such as `User.findById`.
6. **Response to React** JSON plus an HTTP status code travels back to Axios, which updates Redux or local state.

> **SAY THIS IN REVIEW**
>
> Say this in review "A request is a one-way pipeline. It enters Express, passes CORS and parsers, hits a router, optionally passes `isAuth`, reaches a controller, touches MongoDB or the AI service, and comes back as one HTTP response. The response does not walk back through the handlers."

4. Two details that impress reviewers

The route path is assembled from two pieces. `app.use("/api/user", userRouter)` plus `userRouter.get("/current-user", ...)` equals `GET /api/user/current-user`. Always inspect both files before quoting an endpoint.

The AI is not the database. Groq returns text or JSON; the controller decides what to validate and what to save in MongoDB. Nothing is persisted unless controller code saves it.

5. How the three phases connect

Phase	Question it answers	Files
Phase 4	How does a request travel and how is the user authenticated?	`server/index.js`, `server/routes/*`, `isAuth.js`, `user.controller.js`, `user.model.js`
Phase 5	How is data stored, validated and retrieved?	`connectDb.js`, `server/models/*`, `placement.controller.js`
Phase 6	How do the actual product features use all of this?	interview, resume, questions, evaluate, placement pages and controllers

6. Legend used in these notes

Mark	Meaning
`file.js`	Inline code, a file name or a literal value (monospace on a tinted background)
Bold	A defined term, or the key phrase of a sentence
NOTE (blue box)	Context that is important but not obvious from the code alone
FINDING (amber box)	A real issue in this codebase the material for the "what would you improve?" question
SAY THIS IN REVIEW (green box)	A sentence you can say out loud, almost word for word
WATCH OUT (red box)	A common mistake or a risk that costs marks

Every task ends with a green **SAY THIS IN REVIEW** line. If you remember nothing else, remember those fifteen sentences.

II

Backend Request Lifecycle (Phase 4)

WEWITHU REVIEW NOTES

Server bootstrap, route files, JWT middleware, the current-user trace, and HTTP status codes with the real code.

The big picture

Phase 4 answers one question: **what exactly happens between a click in React and a JSON response from Express?** The answer has six stations:

1. **React frontend** `App.jsx` or a page sends an Axios request with a URL, a method and a body.
2. **Express server** `server/index.js` has already registered CORS, parsers and route mounts.
3. **Router** the correct `server/routes/*.js` file matches method + path.
4. **Authentication middleware** `isAuth.js` checks the JWT only on protected routes.
5. **Controller and model** the controller does the work; the model talks to MongoDB.
6. **Response to React** a status code and JSON body return through HTTP.

Not every endpoint uses authentication or the database. Some (for example `POST` `/api/auth/google`) are public; some (for example `POST` `/api/evaluate/written`) only call the AI and never touch MongoDB.

> **NOTE**
> One request, one response A route runs its handlers in order until one of them sends a response or throws. After `res.json(...)` the request is finished; nothing "comes back" through the earlier handlers.

TASK 15

Understand server bootstrap

1. What is server bootstrap?

Bootstrap means **starting and preparing the backend** before it can serve requests. Before WeWithU can answer an API call, its backend must:

- Load configuration values from environment variables.
- Create the Express application object.
- Configure cross-origin requests and request parsing.
- Register (mount) API routes.
- Start listening on a port.

A simple analogy: opening a college office. First you prepare the room, place the desks, put the rules on the wall, tell visitors which counter handles which form, and only then open the door.

2. The actual bootstrap file, line by line

`server/index.js` is short. This is its real order of work:

```
import express from "express"
import dotenv from "dotenv"
import connectDb from "./config/connectDb.js"
import cookieParser from "cookie-parser"
dotenv.config()
import cors from "cors"
import authRouter from "./routes/auth.route.js"
// ...seven more routers imported here

const app = express()
const allowedOrigins = [ /* 5 origins + optional FRONTEND_URL */ ]

app.use(cors({ origin: allowedOrigins, credentials: true }))
app.use(express.json())
```

```
15 app.use(cookieParser())
16
17 app.get("/", (req, res) => { /* small HTML welcome page */ })
18
19 app.use("/api/auth", authRouter)
20 app.use("/api/user", userRouter)
21 app.use("/api/interview", interviewRouter)
22 app.use("/api/payment", paymentRouter)
23 app.use("/api/resume", resumeRouter)
24 app.use("/api/questions", questionsRouter)
25 app.use("/api/evaluate", evaluateRouter)
26 app.use("/api/placement", placementRouter)
27
28 const PORT = process.env.PORT || 6000
29 app.listen(PORT, () => {
30     console.log(`Server running on port ${PORT}`)
31     connectDb()
32 })
```

Each block is explained below. The order matters and is also a review question.

A. dotenv load environment variables

Environment variables hold configuration that must not be hard-coded: the MongoDB connection string, the JWT secret, the Groq API key, Razorpay keys and the port.

```
1 import dotenv from "dotenv";
2 dotenv.config();
```

If `.env` contains `PORT=8000`, then after `dotenv.config()` the value is available as `process.env.PORT`.

Why it is needed: you never want production secrets or machine-specific values inside source code. `.env` is listed in `.gitignore`, so secrets are not committed.

Important detail, and a genuinely impressive one to mention: in ES modules, all `import` statements are evaluated **before** the module body runs, even if the `import` line appears after `dotenv.config()`. So the call on line 5 cannot help modules that read `process.env` at import time. This is exactly why `server/services/razorpay.service.js` calls `dotenv.config()` itself at the top of the file. It is a real, visible consequence of ESM hoisting.

> **WATCH OUT**
>
> Never expose backend secrets to the frontend Vite only exposes variables that start with `VITE_`. A variable named `VITE_JWT_SECRET` would be compiled into the browser bundle and readable by anyone. Secrets stay on the server: `JWT_SECRET`, `MONGODB_URL`, `GROQ_API_KEY`, `RAZORPAY_KEY_SECRET`.

B. Express setup

```
1 import express from "express";
2 const app = express();
```

`express` is the web framework.
`app` is your backend application object.
`app` is used to register middleware, mount routers and start the server.

Creating `app` does not start a server. It only builds the pipeline. Listening happens later in `app.listen()`.

C. CORS allow browser requests from the frontend

CORS means **Cross-Origin Resource Sharing**. The React app runs on `http://localhost:5173` and the backend on another port, so the browser treats them as different origins. Without CORS headers, the browser blocks the response before your JavaScript can read it.

The real configuration in this repository:

```
1 const allowedOrigins = [
2     "http://localhost:5173",
3     "http://localhost:5174",
4     "http://localhost:5175",
5     "https://wewithyou.onrender.com",
6     "https://WeWithU.onrender.com"
7 ];
8 if (process.env.FRONTEND_URL) allowedOrigins.push(process.env.FRONTEND_URL);
9 app.use(cors({ origin: allowedOrigins, credentials: true }));
```

Option	Meaning
`origin`	The list of frontend origins allowed to call this API
`credentials: true`	Allows credentialed cross-origin requests, so the browser will send and store the auth cookie

Three ports are listed because Vite automatically moves to 5174 or 5175 when 5173 is busy a practical detail worth mentioning. `FRONTEND_URL` makes the deployed frontend configurable without a code change.

> **FINDING**
>
> CORS is not authentication CORS only tells the **browser** which origins may read responses. It does not prove who the user is and does not stop `curl`, Postman or a server-side script. Every protected route still needs `isAuth` and every resource needs an ownership check.

Also note the two Render entries differ only in letter case (`wewithyou` and `WeWithU`). Origin comparison is case-sensitive for the host, so one of them is effectively unused. Harmless, but a reviewer may ask why there are two.

D. JSON parsing

```
app.use(express.json());
```

When React sends a JSON body such as:

```
{ "role": "Software Developer", "experience": "Fresher", "mode": "technical" }
```

Express needs to parse it before a controller can read `req.body.role`, `req.body.experience` or `req.body.mode`. Without this middleware, `req.body` is `undefined` and controllers crash on the first property access.
This handles **JSON only**. A PDF upload is `multipart/form-data` and is handled by Multer on the specific route instead. There is no `express.urlencoded()` here, because the frontend always sends JSON or FormData.

E. Cookie parsing

```
app.use(cookieParser());
```

Cookies carry the JWT. Cookie parsing makes them readable through `req.cookies`. In this project the cookie is literally named `token`, so `isAuth.js` reads:

```
let { token } = req.cookies
```

Cookie parsing only **parses** the cookie. It does not validate the token. Verification is a separate step inside `isAuth.js`.

F. Route mounts

A mount connects a URL prefix to a router object:

```
app.use("/api/user", userRouter);
```

The real mounts in this project are:

Prefix	Router file	Responsibility
`/api/auth`	`auth.route.js`	Google sign-in, logout
`/api/user`	`user.route.js`	Current logged-in user
`/api/interview`	`interview.route.js`	Resume parse, question generation, answers, finish, history, report
`/api/payment`	`payment.route.js`	Razorpay order and verification
`/api/resume`	`resume.route.js`	Parse, summarize, ATS check, guidance
`/api/questions`	`questions.route.js`	Placement question generation
`/api/evaluate`	`evaluate.route.js`	Answer and report evaluation
`/api/placement`	`placement.route.js`	Start, save, list and read placement drives

How prefix and router path combine:

Part	Value
Server mount prefix	`/api/user`
Router path	`/current-user`
Full endpoint	`GET /api/user/current-user`

This is why you must open both `server/index.js` and the router file to state an endpoint correctly. There is even a comment in the code explaining this, right below the mounts.

G. listen() start the server

```
const PORT = process.env.PORT || 6000
app.listen(PORT, () => {
    console.log(`Server running on port ${PORT}`)
    connectDb()
})
```

`app.listen()` creates the HTTP server and starts accepting connections on the chosen port. The callback runs once the server is listening.

> **FINDING**
>
> Port inconsistency (a real configuration bug) - Backend default: **6000** (`process.env.PORT || 6000`). - `server/.env.example` sets `PORT=8000`. - The welcome HTML in `server/index.js` says "running successfully on port 8000". - Frontend `serverUrl.js` defaults to **http://localhost:8000** in development.
> So the project only works when `PORT=8000` is actually present in `server/.env`. Without it, the API starts on 6000 while React keeps calling 8000, and every request fails with a network error. Fix: use one source of truth, or set `VITE_API_URL` to match.

> **FINDING**
>
> Database connection starts after listening `connectDb()` is called **inside** the `listen` callback. The server accepts HTTP traffic before MongoDB is known to be connected. A request that arrives during that window can fail with a Mongoose buffering timeout. A more robust order is: connect to the database first, then `app.listen()`, and exit the process if the connection fails.

`connectDb()` also catches its own error and only logs it. After a failed connection the process keeps running with no database, which makes debugging harder. Both details are examined again in Task 20.

3. Why middleware order matters

Express runs middleware **in the order it was registered**, for every request that reaches it.

```
app.use(express.json());
app.use(cookieParser());
app.use("/api/user", userRouter);
```

For `GET /api/user/current-user`, parsing runs before the router, which runs before `isAuth`, which runs before `getCurrentUser`. If a router were mounted before cookie parsing, `req.cookies` would be empty inside `isAuth` and every protected request would look like "no token".

Middleware can do exactly three things:

call `next()` to continue,
send a response to finish the request,
pass an error onward with `next(error)`.

Not every middleware calls `next()` in every situation. `isAuth` calls it only when the token is valid.

> **NOTE**
>
> No 404 or error handler at the end The bootstrap has no final `app.use((req,res) => res.status(404)...)` and no Express error-handling middleware `(err, req, res, next)`. An unknown path therefore returns Express's default HTML 404, and a thrown error outside a controller's own `try/catch` would produce a default HTML 500. Adding both handlers is a standard production improvement.

4. Answers to the five bootstrap questions

1. **What happens when `dotenv.config()` runs?** It reads `.env` and copies the key/value pairs into `process.env`, so `process.env.PORT`, `process.env.JWT_SECRET` and others become available.
2. **Why does WeWithU need CORS?** The frontend and backend are on different origins (different ports locally, different domains in production). CORS headers tell the browser that these origins may read the backend's responses, and `credentials: true` allows the auth cookie.
3. **Difference between `express.json()` and `cookieParser()`?** `express.json()` parses a JSON request body into `req.body`. `cookieParser()` parses the `Cookie` header into `req.cookies`. They handle different parts of the request, and both are needed because this app sends JSON bodies and authenticates with a cookie.
4. **How do prefix and router path combine?** They are concatenated: `/api/user` + `/current-user` = `/api/user/current-user`.
5. **What does `app.listen()` do?** It starts the HTTP server on the given port and begins accepting requests; the callback runs once listening has started.

> **SAY THIS IN REVIEW**
>
> Say this in review "Bootstrap is a fixed order: load env, create app, allow the frontend origin with credentials, parse JSON, parse cookies, mount routers, listen. I also found that the database connects inside the listen callback and that the default port 6000 does not match the frontend default 8000."

TASK 16

Understand route files

1. What is a route?

A route tells Express: **"when a request arrives with this method and this path, run these handlers."**

```
userRouter.get("/current-user", isAuth, getCurrentUser);
```

This single line defines three things:

1. **Method:** `GET`
2. **Path:** `/current-user`
3. **Handlers, in order:** `isAuth` and then `getCurrentUser`

A route file normally does not contain business logic. It is a **mapping table** between incoming requests and the middleware and controllers that should process them.

2. HTTP methods used in this project

Method	Typical purpose	Real examples here
`GET`	Retrieve information	`GET /api/user/current-user`, `GET /api/interview/get-interview`, `GET /api/placement/report/:id`
`POST`	Create something or perform an operation	`POST /api/interview/generate-questions`, `POST /api/payment/order`, `POST /api/evaluate/final-report`
`PATCH`	Partially update a resource	Not used in this project
`PUT`	Replace a resource	Not used in this project
`DELETE`	Delete a resource	Not used; `GET /api/auth/logout` clears the cookie instead

These are conventions, not laws. This codebase is honest about it: logout is a `GET` request, which is unusual and worth flagging (a link prefetch or an image URL could trigger it). Placement completion is a `POST` `/api/placement/save` that both updates and conditionally creates.

3. Follow a route prefix

`server/index.js`:

```
app.use("/api/user", userRouter);
```

`server/routes/user.route.js` :

```
1 userRouter.get("/current-user", isAuth, getCurrentUser);
```

Combined: `GET` `/api/user/current-user` , protected by `isAuth` , handled by `getCurrentUser` .
`isAuth` runs first. If it sends a response, `getCurrentUser` is never called. That "early return" is the whole point of middleware-based authentication.

4. What does middleware inside a route mean?

```
1 router.get("/some-path", middleware, controller);
```

The middleware is just a function that runs as part of the pipeline. It can:

check authentication (`isAuth`),
validate incoming data,
check permissions,
continue with `next()` ,
stop the request by sending a response.

You can list several middlewares. A real example is the interview resume route:

```
1 interviewRouter.post("/resume", isAuth, upload.single("resume"), analyzeResume)
```

Here the order is: verify the user, then accept and store one uploaded file in the field named `resume` , then parse the PDF and call the AI.

5. How to find the handler for any endpoint (the six-step method)

Use this every time; it is also the answer to "how did you trace this feature?"

1. Find the request in the frontend: search for `axios.get(` or `axios.post(` .
2. Record its **method** and **URL**, for example `POST` `/api/interview/submit-answer` .
3. Find the matching `app.use()` prefix in `server/index.js` (`/api/interview`).
4. Open the matching router file (`server/routes/interview.route.js`).
5. Find the route with the same method and the remaining path (`/submit-answer`).
6. Read its middleware and controller, then follow the controller into models and services.

For `GET` `/api/user/current-user` , that trail is:
`client/src/App.jsx` `server/index.js` `server/routes/user.route.js` `server/middlewares/isAuth.js` `server/controllers/user.controller.js` `server/models/user.model.js` .

6. Complete route map (useful in review)

Full endpoint	Method	Middleware	Controller	What it does
`/api/auth/google`	POST	none	`googleAuth`	Find or create user, sign JWT, set cookie
`/api/auth/logout`	GET	none	`logOut`	Clear the `token` cookie
`/api/user/current-user`	GET	`isAuth`	`getCurrentUser`	Return the logged-in user document
`/api/interview/resume`	POST	`isAuth` , `upload.single("resume")`	`analyzeResume`	Read PDF, extract text, AI-structure the resume
`/api/interview/generate-questions`	POST	`isAuth`	`generateQuestion`	Deduct 50 credits, create interview, return questions
`/api/interview/submit-answer`	POST	`isAuth`	`submitAnswer`	AI-evaluate one answer and save it
`/api/interview/finish`	POST	`isAuth`	`finishInterview`	Average scores, mark completed
`/api/interview/get-interview`	GET	`isAuth`	`getMyInterviews`	List the user's interviews
`/api/interview/report/:id`	GET	`isAuth`	`getInterviewReport`	Return one interview report
`/api/payment/order`	POST	`isAuth`	`createOrder`	Create Razorpay order and payment record
`/api/payment/verify`	POST	`isAuth`	`verifyPayment`	Verify signature, mark paid, add credits
`/api/resume/parse`	POST	`isAuth`	`parseResume`	Structure resume text into JSON
`/api/resume/summarize`	POST	`isAuth`	`summarizeResume`	Alias of `parseResume`
`/api/resume/ats-check`	POST	`isAuth`	`atsCheck`	ATS score, keywords, suggestions
`/api/resume/guidance`	POST	`isAuth`	`getPersonalizedGuidance`	Personalized interview-prep report
`/api/questions/generate`	POST	`isAuth`	`generateQuestionsMaster`	Generate questions for any of the 5 test types
`/api/questions/technical`	POST	`isAuth`	`generateTechnical`	Only technical MCQs
`/api/questions/written`	POST	`isAuth`	`generateWritten`	Only written questions

Full endpoint	Method	Middleware	Controller	What it does
`/api/questions/aptitude`	POST	`isAuth`	`generateAptitude`	Only aptitude questions
`/api/questions/softskills`	POST	`isAuth`	`generateSoftskills`	Only soft-skill scenarios
`/api/questions/project`	POST	`isAuth`	`generateProject`	Only project questions
`/api/evaluate/mcq`	POST	`isAuth`	`evaluateMcq`	Score MCQ submissions deterministically
`/api/evaluate/written`	POST	`isAuth`	`evaluateWritten`	AI-evaluate a written answer
`/api/evaluate/softskill`	POST	`isAuth`	`evaluateSoftskill`	AI-evaluate a soft-skill answer
`/api/evaluate/project`	POST	`isAuth`	`evaluateProject`	AI-evaluate a project answer
`/api/evaluate/test-score`	POST	`isAuth`	`getTestScore`	Score one complete test
`/api/evaluate/final-report`	POST	`isAuth`	`getFinalReport`	AI final assessment
`/api/evaluate/overall-score`	POST	`isAuth`	`getOverallScore`	Weighted overall score
`/api/placement/start`	POST	`isAuth`	`startPlacementTest`	Create drive, deduct 50 credits
`/api/placement/save`	POST	`isAuth`	`savePlacementTest`	Update drive or create as fallback
`/api/placement/my-tests`	GET	`isAuth`	`getMyPlacementTests`	List the user's drives
`/api/placement/report/:id`	GET	`isAuth`	`getPlacementTestReport`	Read one drive with ownership check

SAY THIS IN REVIEW

Say this in review "Route files are only dispatch tables. To answer 'what runs for this URL', I combine the mount prefix from `index.js` with the path in the router, then read the middleware list left to right."

TASK 17

Understand authentication middleware

1. First, understand JWT

JWT means **JSON Web Token**. It is a signed token that carries claims, usually a user identifier. A simplified payload:

```
{ "userId": "665f1c2a9b1e4a001234abcd", "iat": 1710000000, "exp": 1710604800 }
```

The payload is **readable** by anyone who has the token. It is signed, not encrypted. Never put secrets in it.
The signature lets the server detect modification.
The token here expires in **7 days** (`jwt.sign(..., { expiresIn: "7d" })` in `server/config/token.js`).

In this project the token travels in an **HttpOnly cookie**, so JavaScript cannot read it and it is sent automatically by the browser on requests that use `withCredentials: true` .

2. The actual middleware, step by step

`server/middlewares/isAuth.js` :

```
const isAuth = async (req, res, next) => {
    console.log("isAuth hit for path:", req.path);
    try {
        let { token } = req.cookies

        if (!token) {
            return res.status(400).json({ message: "user does not have a token" })
        }
        const verifyToken = jwt.verify(token, process.env.JWT_SECRET)

        if (!verifyToken) {
            return res.status(400).json({ message: "user does not have a valid token" })
        }
        req.userId = verifyToken.userId

        next()
    } catch (error) {
        return res.status(500).json({ message: `isAuth error ${error}` })
    }
}
```

Step A read the cookie

```
let { token } = req.cookies
```

The middleware expects a cookie named `token` . `req.cookies` exists because `cookieParser()` is registered in `server/index.js` .

Step B check whether a token exists

If there is no token, it returns `400` and does **not** call `next()` . The controller never runs.

Step C verify the token

```
const verifyToken = jwt.verify(token, process.env.JWT_SECRET)
```

Verification fails if the token is malformed, has a bad signature, has expired, or uses a different secret. `jwt.verify()` **throws** on failure, so the `catch` block handles it.

Step D attach the user id to the request

```
req.userId = verifyToken.userId
```

The controller can now use `req.userId` without parsing the cookie or verifying the token again. This is the bridge between authentication and business logic.

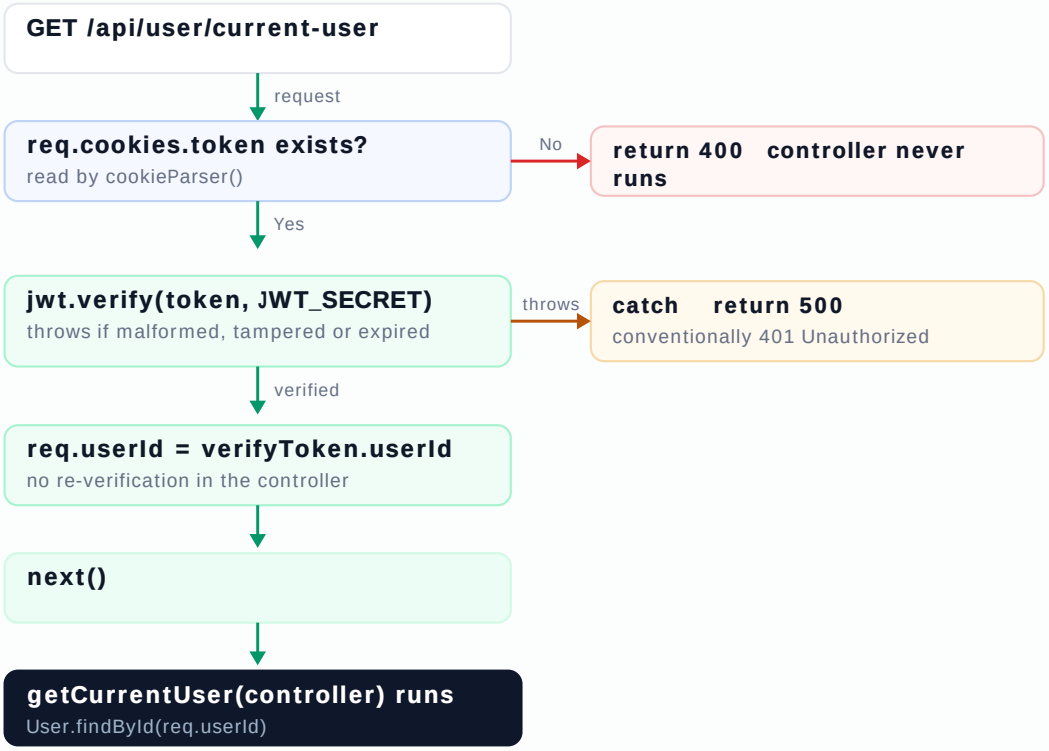
Step E continue

```
next()
```

Express moves to the next handler, here `getCurrentUser`.

> **NOTE**
> The `if (!verifyToken)` check is effectively dead code `jwt.verify()` either returns a decoded payload or throws. It does not normally return a falsy value. So the second `400` branch almost never runs, and an invalid token ends up in `catch` as a `500`. This is a good detail to mention: it shows you read the code rather than the happy path.

3. Visualise the authentication logic



4. Missing, invalid and valid tokens

Situation	What this code does	What it means	Conventional status
Missing token	`400`	No credentials were sent	`401`
Invalid or expired JWT	`jwt.verify` throws, `catch` returns `500`	Credentials could not be verified	`401`
Valid token	Sets `req.userId`, calls `next()`	The next handler may process the request	`200` path
Valid token but no user in DB	Controller returns `404`	Authentication passed, data missing	`404`

> **FINDING**
> Status-code mismatch Missing or invalid credentials are an **authentication failure**, not a server crash. `401` `Unauthorized` is the conventional answer for both. Returning `500` for an expired token also hides real server errors in monitoring and makes the frontend treat a normal logout situation as a crash.

5. Authentication versus authorization

Authentication answers who are you? `isAuth` does this.
Authorization answers are you allowed to do this? ownership checks in controllers.

`isAuth` proves the token is valid. It does not prove the user owns the record they are requesting. In this codebase, `getPlacementTestReport` correctly compares `test.userId` with `req.userId` and returns `403`; `getInterviewReport` and `savePlacementTest` do **not**, which is a real authorization gap (see Part V).

> **SAY THIS IN REVIEW**

> Say this in review "The cookie is parsed by `cookieParser`, `isAuth` verifies the JWT with `JWT_SECRET`, copies `userId` to `req.userId`, and calls `next()`. Invalid tokens currently fall into a catch that returns 500, which should be 401."

TASK 18

Trace the current-user API from frontend to database

1. Why does WeWithU need a current-user API?

When the app opens, the frontend must know **who is logged in**. It needs the user's name, email and credits (used to allow or block starting an interview). The frontend cannot trust a name stored in local state, because anyone can fake it. It asks the backend, and the backend identifies the user from the signed cookie.

2. Step 1 App.jsx sends the request

```
useEffect(() => {
  const getUser = async () => {
    try {
      const result = await axios.get(
        ServerUrl + "/api/user/current-user",
        { withCredentials: true }
      )
      dispatch(setUserData(result.data))
    } catch (error) {
      console.log(error)
      dispatch(setUserData(null))
    }
  }
  getUser()
}, [dispatch])
```

`axios.get` sends an HTTP `GET` request.
`ServerUrl` comes from `client/src/utils/serverUrl.js`: `VITE_API_URL`, else `http://localhost:8000` in development, else `https://wewithyou.onrender.com`.
`withCredentials: true` lets the browser attach the auth cookie to a cross-origin request.
It does **not** create a token. A valid cookie must already exist, which happens after Google sign-in.

3. Step 2 Express receives the request

The request passes the CORS middleware, `express.json()` and `cookieParser()`. Because the path starts with `/api/user`, Express forwards it to `userRouter` mounted at `/api/user`.

4. Step 3 the router matches the endpoint

```
userRouter.get("/current-user", isAuth, getCurrentUser)
```

Full URL: `/api/user/current-user`. `isAuth` runs first, then `getCurrentUser`.

5. Step 4 isAuth verifies the token

1. Reads `req.cookies.token`.
2. Returns an error if the token is missing.
3. Verifies with `process.env.JWT_SECRET`.
4. Copies `verifyToken.userId` to `req.userId`.
5. Calls `next()` on success.

If authentication fails, the controller is never reached.

6. Step 5 the controller retrieves the user

`server/controllers/user.controller.js`:

```
export const getCurrentUser = async (req, res) => {
    try {
        const userId = req.userId
        const user = await User.findById(userId)
        if(!user) {
            return res.status(404).json({message:"user does not found"})
        }
        return res.status(200).json(user)
    } catch (error) {
         return res.status(500).json({message:`failed to get currentUser ${error}`})
    }
}
```

Key lines:

`req.userId` comes from middleware, not from the request body this is why a client cannot request someone else's profile here.
`User.findById(userId)` asks Mongoose to query MongoDB.
No user `404`. Found `200` with the document. Unexpected failure `500`.

The controller does not verify the token again. That work is already done by the route's middleware.

7. Step 6 the User model

`server/models/user.model.js`:

```
const userSchema = new mongoose.Schema({
    name:    { type: String, required: true },
```

```
    email:   { type: String, unique: true, required: true },
    credits: { type: Number, default: 100 }
}, { timestamps: true })
```

Field	Purpose
`name`	User's display name, required
`email`	Login identity, required and unique (unique index)
`credits`	Credit balance, default `100` on signup
`createdAt` , `updatedAt`	Added automatically by `timestamps: true`
`_id`	MongoDB ObjectId, the value stored in the JWT as `userId`

`mongoose.model("User", userSchema)` registers the model. Mongoose turns the model name into the **collection** `users` .

8. Step 7 the frontend receives the response

The JSON body goes into Redux:

```
dispatch(setUserData(result.data))
```

`userSlice.js` stores it as `state.user.userData` . Any component can now read `const { userData } = useSelector(state => state.user)` for example `Step1SetUp` checks `userData.credits < 50` before allowing an interview.
On failure, the current code logs the error and sets user data to `null` . It does not distinguish "not logged in" from "server down". A better version would check `error.response.status` and redirect to `/auth` only for `401` .

9. Complete sequence at a glance

UI	**App.jsx sends GET /api/user/current-user**	withCredentials: true
EX	**CORS express.json() cookieParser()**	origin allowlist + parsed cookies
RT	**mount /api/user + route GET /current-user**	isAuth listed before the controller
AU	**isAuth verifies the JWT**	sets req.userId, calls next()
CT	**getCurrentUser(req, res)**	reads req.userId, no token check
DB	**User.findById(userId)**	Mongoose users collection
RS	**200 + user JSON (or 404 / 500)**	no password or secret is read
RX	**dispatch(setUserData(result.data))**	Redux state.user.userData

The same eight steps in words:

1. `App.jsx` sends `GET` `/api/user/current-user` with credentials.
2. Express applies CORS and parsing, then routes to the user router.
3. `user.route.js` matches `GET` `/current-user` .
4. `isAuth.js` verifies the cookie JWT and sets `req.userId` .
5. `user.controller.js` calls `User.findById(req.userId)` .
6. Mongoose reads the `users` collection in MongoDB.
7. The controller returns JSON with status 200 (or 404/500).
8. Redux stores the returned user data; the UI re-renders.

NOTE

The response travels over HTTP, not backwards through the handlers Many students describe the response as "going back through the model, then the controller, then the middleware". It does not. The controller sends one HTTP response; Axios resolves with that body.

FINDING

Returning the whole document `res.status(200).json(user)` exposes every field of the user document. Today that is only name, email, credits and timestamps, but the pattern becomes risky as soon as sensitive fields are added. A projection like `.select("name email credits")` is safer. The same pattern appears in `verifyPayment` , which returns the whole updated user.

FINDING

Cookie settings are development-only `res.cookie("token", token, { httpOnly: true, secure: false, sameSite: "lax", maxAge: 7*24*60*60*1000 })` . `httpOnly` is excellent. But `secure: false` means the cookie is also sent over plain HTTP, and `sameSite: "lax"` works because frontend and backend are proxied/close in development. In production, the cookie should be `secure: true` , and `sameSite` must be `"none"` if the frontend and backend are on completely different sites otherwise the browser will not send it and every request looks logged out.

SAY THIS IN REVIEW

Say this in review "I can trace one request end to end: Axios in App.jsx with `withCredentials`, the `/api/user` mount, the `/current-user` route, `isAuth` setting `req.userId`, then `User.findById`, then JSON back into Redux. The controller never re-checks the token."

TASK 19

Understand errors and HTTP status codes

An HTTP status code tells the frontend whether a request succeeded, and if not, whose fault it is. The most important distinction is between **a bad request or bad credentials** and **an unexpected server failure**.

1. The five codes you must know

Code	Name	Meaning	Example in this project
400	Bad Request	The request is invalid, incomplete or malformed	`generateQuestion` returns 400 when `role`, `experience` or `mode` is missing
401	Unauthorized	Authentication is missing or invalid	Should be returned for a missing or expired token; currently 400/500
403	Forbidden	Authenticated but not allowed	`getPlacementTestReport` returns 403 when the drive belongs to another user
404	Not Found	The resource does not exist	`getCurrentUser` returns 404 when `findById` finds nothing
500	Internal Server Error	An unexpected server-side failure	Any controller `catch` block; `isAuth` on an invalid JWT

Two codes that are often confused:

400 means "your request itself is wrong" missing fields, wrong types, malformed JSON.
401 means "I cannot verify who you are" no token, expired token, bad signature.
403 means "I know who you are, but you may not do this" another user's report.

`401` and `403` are **not** interchangeable. `401` is about authentication; `403` is about permission.

2. Actual behaviour versus conventional behaviour

For `GET` `/api/user/current-user`:

Scenario	Actual code	Conventional
No token cookie	`400` from `isAuth`	`401`
Expired or invalid JWT	`500` from `isAuth` catch	`401`
Valid token, no user record	`404` from controller	`404`
Valid token, user found	`200` + JSON	`200`
Database or controller error	`500` from controller	`500`

The "conventional" column describes standard API semantics. It does not mean the code already behaves that way.

3. Why the correct code matters

Imagine the frontend receives `500` because the token expired. It may show "server crashed" and offer a retry, when the real fix is to send the user to `/auth` and log in again. Clear codes let the UI:

show the right message,
decide whether to redirect to login,
debug API failures faster,
separate expected auth failures from real server errors in monitoring,
write reliable tests for each error path.

4. Status codes actually used in this project

Controller	Codes you will find	Notes
`auth.controller.js`	400, 200, 500	400 if email missing; 500 for any failure
`user.controller.js`	404, 200, 500	Clean and simple
`interview.controller.js`	400, 404, 500, 200	404 for missing user; finish returns 400 when interview not found (should be 404)
`payment.controller.js`	400, 404, 200, 500	200 for "Already processed" (should be 409 Conflict)
`resume.controller.js`	400, 200, 500	Validation uses a `detail` key, unlike other controllers that use `message`
`questions.controller.js`	400, 200, 500	400 for missing/invalid `test_type`
`evaluate.controller.js`	400, 200, 500	Only MCQ validates its input
`placement.controller.js`	400, 403, 404, 201, 200, 500	`201` on create, `403` on report ownership the best-written controller

FINDING

Response shapes are inconsistent Some controllers return `{ message }`, some return `{ detail }`, some return a bare document, and payment returns `{ success, message, user }`. The frontend therefore handles errors differently per page: `Guidance.jsx` reads `err.response?.data?.detail`, while `PlacementTestPage.jsx` reads `err.response?.data?.message`. A shared error envelope would simplify every page.

5. What to inspect in each controller

For every controller in `server/controllers/`, record:

1. Which status codes it returns.
2. What condition triggers each one.
3. Whether the body has a useful message.
4. Whether exceptions are caught.
5. Whether every path sends exactly one response.
6. Whether errors are logged without leaking secrets.

FINDING

Error objects are sent to the client Several controllers interpolate the whole error object into the response, for example `` `message: failed to get currentUser ${error} `. In Mongoose or JWT errors this can reveal connection details, stack fragments or internal identifiers. Log the full error on the server, and return a short generic message to the client.

SAY THIS IN REVIEW

Say this in review "400 is a bad request, 401 is missing or invalid credentials, 403 is authenticated but not permitted, 404 is missing data, 500 is an unexpected server failure. In this repo the missing-token case returns 400 and an expired token returns 500, so I would normalise both to 401."

III

Database and Data Models (Phase 5)

WEWITHU REVIEW NOTES

MongoDB connection, the four Mongoose schemas, create/read/update lifecycles and the validation layers.

First, understand the architecture

Phase 5 follows one idea:

Controller Mongoose model MongoDB returned document HTTP response

Layer	Job
Frontend	Sends data or asks for data
Route and middleware	Direct the request, check authentication
Controller	Validates input, calls a model method, prepares the response
Mongoose model	Applies schema rules, translates to MongoDB operations
MongoDB	Stores or retrieves documents in a collection

For a **read**, data flows back from MongoDB through Mongoose and the controller to the frontend. For a **write**, the controller may receive the newly created or updated document and return it.

TASK 20

Understand the MongoDB connection

1. What is MongoDB?

MongoDB is a NoSQL **document** database. It stores records in a JSON-like format (BSON) instead of rows and columns.

```
{
  "_id": "665f1c2a9b1e4a001234abcd",
  "name": "Amit",
  "email": "amit@example.com",
  "credits": 100
}
```

The vocabulary:

Database a container for related collections (here, the database in `MONGODB_URL`, e.g. `wewithyou`).
Collection a group of related documents; the equivalent of a table. Mongoose creates `users`, `interviews`, `payments`, `placementtests`.
Document one stored record; the equivalent of a row.
Field one value inside a document.

MongoDB is flexible about document structure. **Mongoose** adds a schema so the application can define and enforce types, required fields, defaults and enums.

2. What does connectDb.js do?

The real file is small and does exactly one job:

```
import mongoose from "mongoose";

const connectDb = async () => {
    try {
        await mongoose.connect(process.env.MONGODB_URL)
        console.log("DataBase Connected")
    } catch (error) {
        console.log(`DataBase Error ${error}`)
    }
}

export default connectDb
```

The important pieces:

`mongoose.connect(...)` starts the connection and creates a connection pool.
`await` waits for the connection attempt to finish, so the `console.log` runs only on success.
`try/catch` logs a failure instead of crashing the process.
The default export lets `server/index.js` call it after `app.listen`.

> **NOTE**
>
> Two differences from the textbook pattern 1. The error is only logged it is **not rethrown** and the process does **not** exit, so the server keeps accepting requests without a database. 2. There is no `mongoose.connection.on("error")` listener, so errors that happen **after** the initial connection (for example Atlas network drops) are not handled explicitly.

3. Which environment variable is needed?

The confirmed name is `MONGODB_URL`:

```
await mongoose.connect(process.env.MONGODB_URL)
```

`server/.env.example` documents it:

```
MONGODB_URL=mongodb+srv://<username>:<password>@cluster0.xxxxx.mongodb.net/wewithyou?retryWrites=true&w=majority
```

The connection string contains credentials. Treat it like a password:

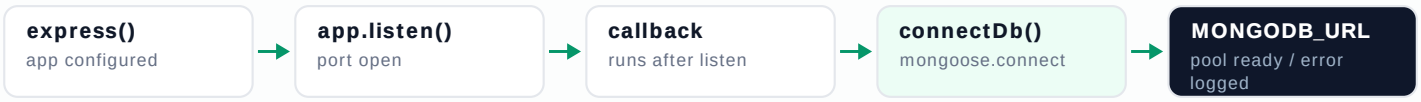
never commit a real `.env` (it is already in `.gitignore`),
never put it in a `VITE_` variable,
give the database user only the permissions it needs,
rotate the password if it is ever exposed.

4. When does the connection start in WeWithU?

`connectDb()` is called **inside the `app.listen()` callback**:

```
app.listen(PORT, () => {
    console.log(`Server running on port ${PORT}`)
    connectDb()
})
```

So the order is:



Warning: HTTP traffic is accepted before the database connection completes a request that needs Mongo can fail while the connection is still opening.

Safer order: await connectDb() app.listen(). connectDb() currently logs the error and does not rethrow.

1. Express is fully configured.
2. `app.listen()` starts accepting requests.
3. The listen callback runs.
4. `connectDb()` begins connecting.
5. The connection succeeds or fails later.

Implication: listening on a port does not mean MongoDB is ready. A request that needs the database can arrive during step 3 5. Mongoose buffers model operations for a while, so requests usually wait rather than fail instantly, but after the buffer timeout they fail with `MongooseError:` `Operation` `buffering` `timed` `out` `after` `10000ms`. A restart can therefore produce a burst of 500s even though the code is correct.

> **SAY THIS IN REVIEW**
>
> Better production order await connectDb() // retry or exit on failure app.listen(PORT, ...) // only accept traffic when the DB is ready For a dashboard, health endpoints can also report `mongoose.connection.readyState`.

5. Debugging checklist for database problems

Is `MONGODB_URL` actually present in `server/.env`?
Is the connection string complete, with a database name and correct password encoding?

Does the MongoDB user have permission on that database?
On Atlas, is the current IP in the network access list? (Render's IPs change; `0.0.0.0/0` is common in demos.)
Did the server log `DataBase Connected` ? If not, read the logged error.
Did the request arrive before the connection was ready?
Is the query using a field that exists in the schema?

> **SAY THIS IN REVIEW**
>
> Say this in review "Mongoose connects with `MONGODB_URL` , but in this codebase the connection starts inside the listen callback, so the API is accepting traffic before the database is ready. The catch only logs the error and does not retry or exit."

TASK 21

Read each Mongoose schema

There are four model files: `user.model.js` , `interview.model.js` , `payment.model.js` , `placement.model.js` .

1. Schema property cheat sheet

Property	Meaning	Watch out for
`type: String`	The value must be a string	Mongoose will cast when it can; `"123"` becomes number 123 for a Number field
`required: true`	A value must be supplied	This is schema validation, not a business rule
`unique: true`	Creates a **unique index**	Not a validator: duplicates cause a MongoDB duplicate-key error, not a normal validation error
`default: 100`	Used when the value is omitted or `undefined`	It does not fix invalid values
`enum: [...]`	Restricts allowed values	Only enforced when validation runs
`ref: "User"`	Declares a reference to another model	Does **not** guarantee the referenced document exists
`timestamps: true`	Adds `createdAt` and `updatedAt`	Automatically managed by Mongoose

2. User model `user.model.js`

```
const userSchema = new mongoose.Schema({
    name:    { type: String, required: true },
    email:   { type: String, unique: true, required: true },
    credits: { type: Number, default: 100 }
}, { timestamps: true })
```

Field	Rules
`name`	String, required
`email`	String, required, unique index
`credits`	Number, default `100`
`createdAt` , `updatedAt`	Automatic

Example: a user created as `{ name: "Amit", email: "amit@example.com" }` receives `credits: 100` . Note that `googleAuth` passes `credits: 100` explicitly as well, which is redundant but harmless.

3. Interview model `interview.model.js`

This model stores one interview session together with its questions and results.

Field	Type and rules
`userId`	ObjectId, `ref: "User"` , required
`role`	String, required
`experience`	String, required
`mode`	String, required, enum `technical` or `hr`
`resumeText`	String, optional
`questions`	Array of embedded question documents
`finalScore`	Number, default `0`
`status`	String, default `Incompleted` , enum `Incompleted` or `completed`
`createdAt` , `updatedAt`	Automatic

Each embedded question has its own schema (`questionsSchema`):

Field	Purpose	Default
`question`	The interview question	
`difficulty`	Difficulty chosen at setup	
`timeLimit`	Seconds allowed for this question	

Field	Purpose	Default
`answer`	Candidate's answer	
`feedback`	Feedback for that answer	
`score`	Overall score for the answer	`0`
`confidence`	Confidence sub-score	`0`
`communication`	Communication sub-score	`0`
`correctness`	Correctness sub-score	`0`

`questions` is an array of embedded documents, not references. The question records live inside the interview document and are loaded with it in a single read. This is a deliberate denormalisation: questions are never shared between interviews, so embedding is simpler and faster than a separate `Question` collection.

An illustrative interview document:

```
{
  "role": "Backend Developer",
  "experience": "Fresher / 0 Years Experience",
  "mode": "technical",
  "questions": [
    {
      "question": "What is an API?",
      "answer": "An interface that allows systems to communicate.",
      "score": 8,
      "feedback": "Good explanation"
    }
  ],
  "finalScore": 8,
  "status": "completed"
}
```

This is a shape illustration, not a real stored record.

4. Payment model `payment.model.js`

Field	Type and rules
`userId`	ObjectId, `ref: "User"`, required
`planId`	String, optional
`amount`	Number, optional
`credits`	Number, optional
`razorpayOrderId`	String, optional
`razorpayPaymentId`	String, optional
`status`	String, enum `created`, `paid`, `failed`, default `created`
`createdAt`, `updatedAt`	Automatic

A payment record tracks the order id, the payment id (filled in after verification), the amount, how many credits were bought, and the lifecycle state.

The enum prevents arbitrary strings **through normal Mongoose validation**. It cannot verify that a payment really happened that is the HMAC signature check in `verifyPayment`.

5. Placement model `placement.model.js`

This is the richest model; it stores a complete five-module drive.

Field	Type and rules
`userId`	ObjectId, `ref: "User"`, required
`candidateName`	String, required
`targetRole`	String, required
`level`	String, required
`difficultyMode`	String, default `medium`
`resumeText`	String, optional
`resumeData`	Nested object: name, skills[], projects[], experience_level, education, summary
`testScores`	Array of score objects
`allStrengths`	Array of strings
`allWeaknesses`	Array of strings
`finalReport`	Nested final assessment object
`status`	String, enum `Incompleted` / `completed`, **default `completed`**
`createdAt`, `updatedAt`	Automatic

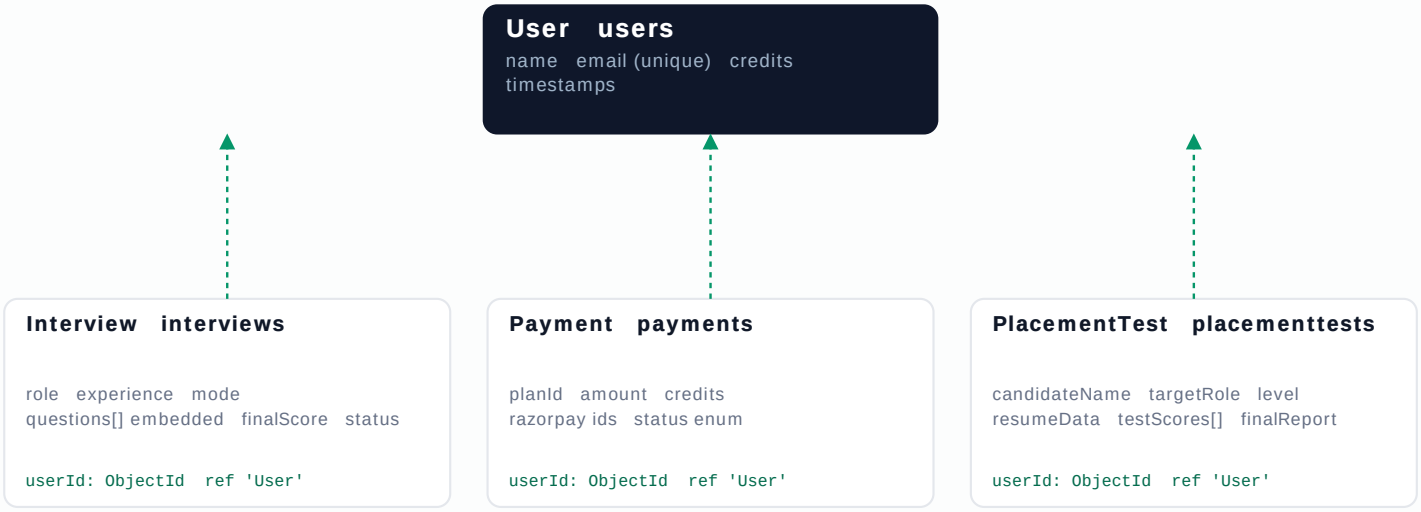
Each entry in `testScores` can contain:

Field	Meaning
`test_name`	Display name, e.g. "Technical MCQs"
`test_type`	Key, e.g. `mcq_technical`
`score`	Raw score
`total`	Maximum
`percentage`	Percentage
`score_out_of_10`	Normalised score used for weighting

`finalReport` can contain: `overall_score`, `overall_percentage`, `rating`, `summary`, `positive_areas`, `negative_areas`, `study_plan[{topic, priority, resource}]`, `readiness_score`, `estimated_ready_in`.

> **NOTE**
> The status default is misleading The schema says `status` defaults to `"completed"`, yet `startPlacementTest` explicitly creates the document with `status: "Incompleted"`. Explicit values always win over defaults. If someone later creates a placement document without specifying status, it will silently appear completed. This is a design smell worth mentioning: the default should probably be `"Incompleted"`.

6. How the four models relate



References are logical only: MongoDB enforces no foreign key, so ownership must be checked in the controller.

`Interview`, `Payment` and `PlacementTest` each store a `userId` with `ref: "User"`. That connects each record to its owner **logically**, but MongoDB does not enforce it like a relational foreign key:

deleting a user does not delete their interviews,
inserting an interview with a random ObjectId succeeds,
`populate()` is available but is not used here; controllers just use the stored id,
ownership must be checked in the controller and in `getPlacementTestReport` it is.

> **SAY THIS IN REVIEW**
> Say this in review "Four collections: users with name/email/credits, interviews with embedded question documents, payments with a status enum, and placement tests with nested resume data, test scores and a final report. The three child models reference User by ObjectId, but ownership is only enforced in controller code."

TASK 22

Trace a create, read and update operation

The placement feature is the best example because it has all three operations.

A. Create startPlacementTest

`server/controllers/placement.controller.js`, step by step:

1. `User.findById(req.userId)` find the logged-in user.
2. If no user `404`.
3. If `user.credits` `<` `50` `400`.
4. Destructure `candidateName`, `targetRole`, `level`, `difficultyMode` from `req.body`.
5. If any of the first three is missing `400`.
6. `PlacementTest.create({...})`.
7. `user.credits` `-=` `50;` `await` `user.save()`.
8. Return `201` with `creditsLeft` and `testId`.

The key call:

```
const newTest = await PlacementTest.create({
  userId: user._id,
  candidateName,
  targetRole,
  level,
  difficultyMode: difficultyMode || "medium",
  status: "Incompleted",
  testScores: []
});
```

`PlacementTest.create()` builds the document and saves it in one step.
Mongoose fills `_id`, `createdAt` and `updatedAt`.
The returned `newTest._id` is sent to the frontend as `testId`, which becomes `activeTestId` and is reused by every later request.

FINDING

Two writes, no transaction Creating the test and deducting credits are separate operations. If the second fails (network drop, validation error), the user keeps the test without paying. If the first fails after the deduction... in this code the deduction happens after creation, so the realistic failure is "test created, credits not deducted". A MongoDB transaction, or a single atomic update with a guard, removes the inconsistency.

B. Read getMyPlacementTests

```
const tests = await PlacementTest.find({ userId: req.userId })
  .sort({ createdAt: -1 })
  .select("targetRole level finalReport createdAt status");
```

Part	Meaning
`find({ userId: req.userId })`	Only this user's drives; the filter is the authorization
`.sort({ createdAt: -1 })`	Newest first
`.select("...")`	Return only the fields the history page needs
`await`	Wait for the query to run

The result is returned as JSON. This is good practice: the history list does not need `resumeText`, `testScores` or every nested report field, so the response stays small.

C. Update savePlacementTest

1. Find the user (`404` if missing).
2. Destructure the full payload from `req.body`.
3. Require `candidateName`, `targetRole`, `level`, `testScores`, `finalReport` else `400`.
4. If `testId` is present, `PlacementTest.findById(testId)`.
5. If found, assign every field, set `status = "completed"`, then `await test.save()`.
6. If no test was found, a **fallback** creates a new document and deducts 50 credits.
7. Return `200` with `creditsLeft` and `testId`.

```
test = await PlacementTest.findById(testId);
if (test) {
  test.testScores = testScores;
  test.finalReport = finalReport;
  test.status = "completed";
  await test.save();
}
```

`test.save()` writes only the modified paths and runs schema validation on them unlike `findByIdAndUpdate`, which by default does **not** run full document validation.

FINDING

Missing ownership check on update The controller looks up the test by id but never compares `test.userId` with `req.userId`. Any authenticated user who knows another user's test id can overwrite their scores and report by calling `POST /api/placement/save`. The read endpoint does check ownership (`403`), so the fix is small: apply the same comparison before assignment.

FINDING

The fallback double-charges If a client sends a wrong or stale `testId`, the code silently creates a **new** placement drive and deducts another 50 credits. A missing id should be a `400` or `404`, not a new charge. This is also why the frontend must keep `activeTestId` in state for the whole session.

D. The complete data lifecycle



The steps run in order; the HTTP response is sent once at the end (201 on create, 200 on update).

Finding: the update step looks the test up by id but never compares test.userId with req.userId.

1. **Create** `PlacementTest.create(...)` makes the drive and `user.save()` deducts credits.
2. **Read** `PlacementTest.find({ userId })` powers the history list.
3. **Update** `findById` assign fields `save()` stores scores and the final report.
4. **Respond** an HTTP status plus JSON body reaches the frontend, which navigates to `/placement-results/:id`.

E. Mongoose methods to recognise

Method	What it does	Notes
`Model.create(data)`	Creates and saves a document	Runs schema validation
`Model.find(filter)`	Finds many documents	Returns an array, possibly empty
`Model.findOne(filter)`	Finds one document	Used in `googleAuth` and `verifyPayment`
`Model.findById(id)`	Finds one by `_id`	Throws `CastError` for a malformed id unless caught
`document.save()`	Persists changes	Runs validation on modified paths
`Model.findByIdAndUpdate(id, update, opts)`	Update in one call	`{ new: true }` returns the updated doc; validators are off by default
`Model.findOneAndUpdate(filter, update, opts)`	Same, with any filter	Used with `$inc` to add credits

In `verifyPayment`, `User.findByIdAndUpdate(payment.userId, { $inc: { credits: payment.credits } }, { new: true })` is a good example: `$inc` is atomic on the server, so two simultaneous verifications cannot lose an increment. (They can still double-add if both pass the `status === "paid"` check before either saves an idempotency key or a conditional update would close that window.)

> **SAY THIS IN REVIEW**
>
> Say this in review "Placement drives show all three operations: create with credits deducted, read with a `userId` filter and a projection, and update through `findById` plus `save()`. I also found that the update path does not verify ownership and that a bad test id silently creates a second charge."

TASK 23

Understand data validation

Validation means checking that incoming data is acceptable **before** the app relies on it or stores it. WeWithU validates at several layers, and each layer owns a different kind of rule.

1. Controller-level validation (business rules)

Real examples:

```
// placement.controller.js
if (!candidateName || !targetRole || !level) {
  return res.status(400).json({ message: "Missing required details to start exam." });
}
```

```
// interview.controller.js
if (!role || !experience || !mode) {
  return res.status(400).json({ message: "Role, Experience and Mode are required." });
}
if (user.credits < 50) {
  return res.status(400).json({ message: "Not enough credits. Minimum 50 required." });
}
```

```
// payment.controller.js
if (!amount || !credits) {
  return res.status(400).json({ message: "Invalid plan data" });
}
```

Controller validation is where these rules belong:

required inputs are present,
the user has enough credits,
the current user owns the resource,
a payment is verified before credits are granted,
a resume has projects before the project test is attempted.

2. Schema-level validation (document structure)

```
name:  { type: String, required: true }
email: { type: String, unique: true, required: true }
mode:  { type: String, enum: ["technical", "hr"], required: true }
```

Mongoose rejects a save when a required field is missing or an enum value is outside the list. It will also cast types where possible: a numeric string becomes a number for a Number field.
Schema validation is a **safety net**, not a replacement for controller checks. A schema can confirm that `credits` is a number; it cannot confirm that the user is allowed to spend it.

3. Duplicate handling

`email` has `unique: true`, which creates a unique index in MongoDB. If two users try to register with the same email:
Mongoose does not catch it as a normal `ValidationError`,
MongoDB raises a **duplicate-key error** (code `11000`),
`googleAuth`'s `catch` returns `500`.

> **FINDING**
> Duplicate email returns 500 The correct response is `409` `Conflict`. The fix is a small check in the catch block: if (error.code === 11000) { return res.status(409).json({ message: "Email already registered" }); }

4. What this project currently validates

Layer	Examples in WeWithU	What it protects
Frontend	`Step1SetUp` blocks start without role/experience/mode and when `credits < 50`; `PlacementTestPage` checks name and credits; PDF type checks	User experience, fewer wasted requests
Controller	Required placement details, credits, required interview fields, `amount/credits` presence	Business rules and server-side authority
Mongoose schema	Required fields, types, enums, defaults, `timestamps`	Document shape and consistency
MongoDB index	Unique email index	Prevents duplicate accounts
Auth / authorization	JWT verification, placement report ownership check	Access control per user

Frontend validation is **not** security. Anyone can call the API directly with Postman or `curl`, so every important check must exist on the server. This project does repeat the important checks server-side, which is the right pattern.

5. Validation gaps worth showing a reviewer

1. **Missing values** `evaluateWritten`, `evaluateSoftskill`, `evaluateProject` do not check that `question` or `answer` exist. Empty input is sent straight to the AI.
2. **Wrong types** `numQuestions` and `question_count` are not range-checked. `parseInt` is applied in `generateQuestionsMaster`, but a huge value would still create a huge AI prompt and cost.
3. **Invalid values** `difficulty` is used in prompts but not restricted to `easy|medium|hard|expert`; an unexpected value falls back to a 90-second timer and a capitalised label.
4. **Duplicates** duplicate email produces `500` instead of `409`.
5. **Ownership** `savePlacementTest` and `getInterviewReport` do not verify ownership; `getPlacementTestReport` does.
6. **Malformed ObjectIds** `findById("not-an-id")` throws a `CastError`, which lands in `catch` and becomes a `500` instead of `400`.
7. **Credits** deduction and creation are separate writes with no transaction; concurrent requests can both pass the `credits < 50` check.
8. **Score ranges** AI-provided `confidence`, `communication`, `correctness` and `score` are saved without clamping to 0 10. If the model returns 12 or `"8"`, the database stores it as-is (Mongoose will cast numeric strings, but 12 stays 12).
9. **Payment trust** `createOrder` accepts `amount` and `credits` **from the request body** and forwards them to Razorpay and to the payment record. A client can send `{ amount: 1, credits: 650 }` and buy 650 credits for one rupee. Prices and credit grants must come from a server-side plan table keyed by `planId`.

> **WATCH OUT**
> Why `!amount || !credits` is not enough It rejects `0`, `""`, `null` and `undefined`, but it accepts `-100` and `"abc"`. It also does not verify that the amount matches the chosen plan or that the credit amount is one of the real packages. Presence checks are not value checks.

> **SAY THIS IN REVIEW**
> Say this in review "Validation is layered: frontend for UX, controller for business rules and ownership, Mongoose for types/required/enums/defaults, and the unique index for email. The gaps are ownership on update, duplicate-key mapping to 409, and trusting client-supplied payment amounts."

IV

Core Product Features (Phase 6)

WEWITHU REVIEW NOTES

Interview workflow, AI prompts, resume flows, the five-module placement drive, history, reports and static pages.

Phase 6 connects everything: how the real product works, feature by feature, from click to database and back.

The overall architecture

React UI pages components Redux state

Axios ServerUrl + /api/... withCredentials

Express + middleware cors express.json() cookieParser() 8 routers

isAuth **controller** JWT verify validation askAi() Mongoose calls

AI service / MongoDB Groq chat completions collections: users, interviews, payments, placementtests

one request flows top to bottom; the response returns over HTTP

Layer	What happens
React UI	User enters details, answers questions or uploads a resume
Axios + API route	Sends data to the matching Express endpoint
Controller	Validates input, calls AI services or database models
AI service	`askAi()` in `groq.service.js` generates questions, feedback or reports
MongoDB	Stores sessions, scores and reports
Response and UI update	React shows questions, feedback, history or reports

Not every feature uses both AI and MongoDB:

- `POST` `/api/evaluate/written` uses AI and writes nothing to the database directly.
- `GET` `/api/interview/get-interview` reads MongoDB and calls no AI.
- `GET` `/api/auth/logout` touches neither.
- `CompanyPrep.jsx` and `Roadmap.jsx` are pure frontend pages.

TASK 24

Interview workflow

Files to follow: `InterviewPage.jsx`, `Step1SetUp.jsx`, `Step2Interview.jsx`, `InterviewHistory.jsx`, `InterviewReport.jsx`, `interview.route.js`, `interview.controller.js`, `interview.model.js`.

1. The user's journey

1 Set up
role, experience, mode, difficulty
`Step1SetUp`

→

2 Questions
POST /generate-questions
50 credits interview
`generateQuestion`

→

3 Answers
POST /submit-answer
AI scores save
`submitAnswer`

→

4 Finish
POST /finish
averages completed
`finishInterview`

→

5 Report
GET /report/:id
history + report pages
`getInterviewReport`

Answering the last question does not create a report by itself: /finish is what writes finalScore and status.

Leftover state: interviewData lives in Redux, so the session survives a route change but not a full reload.

1. **Set up** role, experience, mode, number of questions, difficulty, optional resume.
2. **Generate questions** `POST` `/api/interview/generate-questions`.
3. **Answer** each answer goes to `POST` `/api/interview/submit-answer`.
4. **Finish** `POST` `/api/interview/finish`.
5. **Review** history and report pages read saved data.

2. Setup and resume analysis Step1SetUp.jsx

State kept by the component: `role`, `experience` (default empty), `mode` (`technical`), `numQuestions` (`5`), `difficulty` (`medium`), `resumeFile`, `projects`, `skills`, `resumeText`, `analysisDone`, `analyzing`, `loading`, `error`.
Experience options are a fixed list from fresher to 10+ years. Roles are searched against `client/src/utils/jobRoles.js`, and the first eight matches are suggested.
When the user drops or selects a PDF:

```
const formdata = new FormData()
formdata.append("resume", file)
const result = await axios.post(ServerUrl + "/api/interview/resume", formdata, { withCredentials: true })
```

The response fills `role`, `experience`, `projects`, `skills` and `resumeText`, and `analysisDone` flips the card to the "Resume Uploaded Successfully" view. Only `application/pdf` is accepted in the drop handler.
`FormData` is used because the body is a **file upload** (`multipart/form-data`), not JSON. In `FormData` you must **not** set `Content-Type` manually; the browser adds the multipart boundary.

3. Generate the interview questions

```
const result = await axios.post(
  ServerUrl + "/api/interview/generate-questions",
  { role, experience, mode, resumeText, projects, skills, numQuestions, difficulty },
  { withCredentials: true }
)
```

On success the component does three things:

```
if (userData) dispatch(setUserData({ ...userData, credits: result.data.creditsLeft }))
dispatch(setInterviewData(result.data))
navigate("/interview-session")
```

The payload returned by the backend is:

```
{ "interviewId": "...", "creditsLeft": 50, "userName": "Amit", "questions": [ ... ] }
```

That whole object becomes `interview.interviewData` in Redux, so `Step2Interview` can read `interviewId`, `questions` and `userName` without prop drilling.
On the backend (`generateQuestion`):

1. Trim and validate `role`, `experience`, `mode` `400` if missing.
2. `User.findById(req.userId)` `404` if no user.
3. `user.credits` `<` `50` `400`.
4. Build a system prompt (role-play as an interviewer, exact question count, strict difficulty, strict JSON shape) and a user prompt (role, experience, mode, projects, skills, resume).
5. `askAi(messages,` `{` `json:` `true,` `returnParsed:` `true` `})`.
6. Validate `aiResponse.questions` is a non-empty array else `500`.
7. `user.credits` `-=` `50;` `await` `user.save()`.
8. `Interview.create({` `userId,` `role,` `experience,` `mode,` `resumeText,` `questions:` `[...]` `})` with a per-difficulty `timeLimit` (easy 60 s, medium 90 s, hard 120 s, expert 150 s).
9. Return `interviewId`, `creditsLeft`, `userName`, `questions`.

> **NOTE**
> Credits are deducted only after the AI succeeds This ordering is a genuine design strength: a failed AI call does not charge the user. The cost is that a slow AI call keeps the user waiting before the credit write, which is acceptable here.

4. Submit answers and receive feedback Step2Interview.jsx

The component runs the whole voice interview:

`window.speechSynthesis` reads questions aloud; it picks a female voice (Zira / Samantha) if available, else a male one (David / Mark), else the first voice, and swaps the assistant video accordingly.
`webkitSpeechRecognition` (Chrome-only) appends the transcript to the answer.
A `setInterval` timer counts down from `currentQuestion.timeLimit`, and when it reaches zero `submitAnswer()` fires automatically.

```
const result = await axios.post(ServerUrl + "/api/interview/submit-answer", {
  interviewId,
  questionIndex: currentIndex,
  answer,
  timeTaken: currentQuestion.timeLimit - timeLeft,
}, { withCredentials: true })
setFeedback(result.data.feedback)
speakText(result.data.feedback)
```

`questionIndex` identifies which embedded question is being answered; `interviewId` identifies the session.
On the backend (`submitAnswer`):

1. Load the interview by id, then pick `interview.questions[questionIndex]`.
2. If `answer` is empty score `0`, feedback "You did not submit an answer.", save, return.
3. If `timeTaken` `>` `question.timeLimit` score `0`, feedback "Time limit exceeded. Answer not evaluated.", save, return.

4. Otherwise build an evaluation prompt asking for `confidence`, `communication`, `correctness`, `finalScore` (their average) and a 10 15 word feedback line.
5. `askAi(...)`, then write all five values onto the embedded question and `interview.save()`.
6. Return `{ feedback }`.

> **FINDING**
>
> Missing checks in `submitAnswer` There is no `if (!interview)` check, no bounds check on `questionIndex`, and no ownership check on `interview.userId`. A deleted interview or an out-of-range index throws a `TypeError` that becomes a `500`. Any logged-in user could submit answers into another user's interview if they knew the id.

5. Finish the interview

```
const result = await axios.post(ServerUrl + "/api/interview/finish", { interviewId }, { withCredentials: true })
onFinish(result.data)
```

On the backend (`finishInterview`):

1. `Interview.findById(interviewId)` `400` if not found (should be `404`).
2. Sum `score`, `confidence`, `communication`, `correctness` over all questions.
3. Divide each total by the number of questions.
4. Save `interview.finalScore` and set `interview.status = "completed"`.
5. Return `finalScore`, the three averages (rounded to one decimal) and `questionWiseScore`.

> **NOTE**
>
> Reaching the last question does not save anything by itself The report exists in the database only after `/finish` runs. If the user closes the tab after answering everything but before the last "next", the interview stays `Incompleted` with `finalScore: 0`. The `beforeunload` warning and the custom leave modal in `InterviewPage.jsx` exist precisely to prevent that.

6. Where is the interview stored?

In one `interviews` document: `userId`, `role`, `experience`, `mode`, `resumeText`, an array of embedded questions (each with its answer, feedback and four scores), `finalScore`, `status`, and timestamps.

> **SAY THIS IN REVIEW**
>
> Say this in review "Setup posts a PDF as FormData, question generation deducts 50 credits and creates the interview document, each answer is AI-scored and saved inside that document, and `finish` averages the scores and marks it completed. Only then can the history page show it."

TASK 25

AI question generation and evaluation

1. Where does the AI functionality live?

Everything goes through one shared service:

```
import { askAi } from "../services/groq.service.js";
```

Controllers never call Groq directly. This centralisation gives one place for the API key, model name, JSON mode, JSON extraction and retry logic.

`askAi(messages, options, retries)` in `server/services/groq.service.js`:

Step	Detail
Validate input	Throws if `messages` is not a non-empty array
Build payload	`model = process.env.GROQ_MODEL` or `llama-3.3-70b-versatile`, `messages`, `temperature` (default `0.7`)
Optional limits	`max_tokens` is added only when supplied
JSON mode	Sets `response_format: { type: "json_object" }` when `options.json` is true **or** any message text contains the word "json"
HTTP call	`axios.post("https://api.groq.com/openai/v1/chat/completions", ...)` with `Authorization: Bearer ${GROQ_API_KEY}`
Empty check	Throws if the content is empty or whitespace
JSON extraction	`extractJson()` handles raw JSON, fenced code blocks, and the first `{...}` / `[...]` region
Return	Returns the parsed object when `returnParsed: true`, otherwise the string
Rate limits	On HTTP 429, reads "try again in Xs" and retries up to 3 times with a delay
Errors	Throws `Groq API Error: <provider message>`

`extractJson()` deserves a mention in the review because it is defensive engineering: LLMs sometimes wrap JSON in explanations or markdown fences, and this helper recovers from all three cases.

2. Prompt-based question generation

The controller builds **two messages**.

System message (rules for the model):

act as a real human interviewer speaking simple, natural English,
generate exactly `numQuestions` questions,
conform strictly to the difficulty (`expert` is mapped to "Expert / Advanced"),
return strictly `{ "questions": ["...", "..."] }` ,
each question 15 25 words, one sentence, unnumbered, no explanations,
base questions on role, experience, mode, projects, skills and resume.

User message (candidate data):

```
Role: Backend Developer
Experience: Fresher / 0 Years Experience
InterviewMode: technical
Projects: Library Management System, Chat App
Skills: Node.js, Express, MongoDB,
Resume: ...
```

Then:

```
const aiResponse = await askAi(messages, { json: true, returnParsed: true })
```

`json: true` ask Groq for JSON mode.
`returnParsed: true` return an object instead of a string.

After the call the controller:

1. verifies `aiResponse.questions` is an array (otherwise `500` "AI returned empty or invalid response"),
2. trims each question and drops empty strings,
3. slices to `numQuestions` ,
4. rejects an empty result (`500` "AI failed to generate questions"),
5. deducts credits and creates the interview.

The placement question controller adds an extra layer: `parseJsonArray()` in `questions.controller.js` accepts an array, `{questions: [...]}` , a fenced block, or any object containing an array, and wraps a single question object in an array. It is deliberately forgiving because the same shape is not guaranteed on every call.

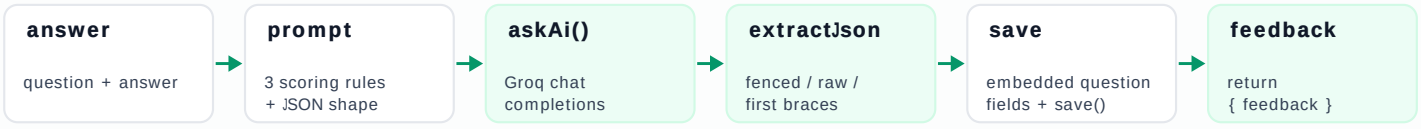
3. How the AI evaluates an answer

The prompt contains:

the question,
the candidate's answer,
definitions of confidence, communication and correctness (0 10),
the rule `finalScore = average of the three, rounded` ,
feedback rules (10 15 words, human tone, no repetition of the question),
the exact JSON shape:

```
{
  "confidence": 8,
  "communication": 7,
  "correctness": 9,
  "finalScore": 8,
  "feedback": "Your answer is relevant and clear, but include a more specific example."
}
```

The controller writes those values onto `interview.questions[questionIndex]` and calls `interview.save()` , then returns only `{ feedback }` .



Generation validates that questions is a non-empty array. Evaluation does not validate or clamp the five returned numbers.

AI output is external input: parse it, check the structure, clamp ranges, and treat provider failure as a normal error path.

4. What happens if the AI response is wrong?

Controller	Guard present	Risk
`generateQuestion`	Checks `questions` is a non-empty array	Good
`submitAnswer`	**No validation** of `parsed.confidence` , `communication` , `correctness` , `finalScore` , `feedback`	Missing fields store `undefined` ; out-of-range values (e.g. 15) are stored as-is
`generateQuestionsMaster`	`parseJsonArray` throws a clear error	Good
`evaluateWritten/Softskill/Project`	None; the AI object is returned as-is	Frontend reads `feedback.score` , so a missing `score` pushes `undefined` into `testAnswers`
`getFinalReport`	None	Missing `final_score` fields produce an empty-looking report

NOTE

> The principle to state out loud **AI output is external input.** Even when you ask for JSON, you must parse it, validate its structure and values, clamp numbers to the expected range, and handle provider failures. A generated score is an estimate from a language model, not an objective measurement.

5. AI layer versus database layer

AI service	Database layer
Generates questions	Stores the generated questions
Evaluates answers	Stores answers and scores
Writes feedback text	Makes feedback available to later report requests
Can fail, time out or return invalid JSON	Persists only what the controller decides to save

The AI never writes to MongoDB. The controller is the only component that decides what to persist.

6. Prompts in this project, at a glance

Feature	Controller	Prompt focus
Interview questions	`generateQuestion`	Role-play interviewer, strict JSON, 15 25 words per question
Answer evaluation	`submitAnswer`	Three sub-scores + average + short feedback
Resume structure	`analyzeResume` , `parseResume`	Return role/experience/projects/skills or the placement profile
ATS check	`atsCheck`	`ats_score` , matched/missing keywords, suggestions, verdict
Guidance	`getPersonalizedGuidance`	skills checklist, project talking points, technical and HR tips
Placement questions	`getTechnicalMCQs` , `getWrittenQuestions` , `getAptitudeQuestions` , `getSoftSkillQuestions` , `getProjectQuestions`	One prompt per test type with its own JSON schema
Written/soft/project evaluation	`evaluateWritten` , `evaluateSoftskill` , `evaluateProject`	Score, verdict, strengths, weaknesses, suggestion
Final report	`getFinalReport`	Overall score, rating, summary, study plan, readiness

> **SAY THIS IN REVIEW**
>
> Say this in review "All AI calls go through `askAi` , which handles the model, JSON mode, JSON extraction and 429 retries. I compared the generation path, which validates its output, with the evaluation path, which does not clamp or check scores."

TASK 26

Resume workflow

WeWithU has **four** resume-related flows. They must not be confused: uploading a PDF, parsing text, checking ATS compatibility and generating career guidance.

1. Interview resume upload a file upload

Route:

```
interviewRouter.post("/resume", isAuth, upload.single("resume"), analyzeResume)
```

Part	What it does
`isAuth`	Verifies the cookie JWT
`upload.single("resume")`	Accepts one file in the field named `resume`
`analyzeResume`	Reads the file, extracts text, calls the AI

Multer configuration (`server/middlewares/multer.js`):

```
const storage = multer.diskStorage({
  destination: (req, file, cb) => cb(null, "public"),
  filename: (req, file, cb) => cb(null, Date.now() + "-" + file.originalname)
})
export const upload = multer({ storage, limits: { fileSize: 5 * 1024 * 1024 } })
```

Files are stored on disk in `server/public` with a timestamp prefix to avoid collisions.
The limit is **5 MB**.
There is **no MIME-type or extension filter** in Multer, so the "PDF only" rule is enforced only in the browser. The server relies on `pdfjs-dist` failing for non-PDFs.

Controller flow in `analyzeResume` :

1. `if (!req.file) return 400 "Resume required"` .
2. Read the file into a buffer, convert to `Uint8Array` .
3. `pdfjsLib.getDocument({ data })` and loop over pages, joining `item.str` values.
4. Normalise whitespace: `replace(/\s+/g, " ").trim()` .
5. Prompt the AI to return `{ role, experience, projects, skills }` as strict JSON.
6. `askAi(messages, { json: true, returnParsed: true })` .
7. `fs.unlinkSync(filepath)` delete the temporary file **after successful processing**.

8. Respond with `role`, `experience`, `projects`, `skills` and the full `resumeText`.
9. In `catch`, if the file still exists, attempt `fs.unlinkSync` again, then return `500` with `error.message`.

> **FINDING**
>
> Filename and cleanup details The stored name is `Date.now()` `+` `"-"` `+` `file.originalname`, so the client's original filename (including any path-like characters it contains) is part of the on-disk name. A sanitised name and an allow-list of `application/pdf` would be safer. Also, `fs.unlinkSync` runs **before** the response is sent, so a failure to delete throws inside the `try` and turns a successful analysis into a `500`.

2. Resume parsing for placement tests text, not a file

`PlacementTestPage.jsx` extracts text **in the browser** using PDF.js loaded from a CDN (`window.pdfjsLib`, worker `pdf.worker.min.js` 3.11.174), then sends the text:

```
await axios.post(ServerUrl + "/api/resume/parse", {
  resume_text: resumeText,
  target_role: form.role,
}, { withCredentials: true })
```

`parseResume` in `resume.controller.js`:

1. Requires `resume_text` `400` `{` `detail:` `"resume_text` `is` `required."` `}`.
2. `cleanText()` strips control characters and collapses whitespace.
3. `substring(0,` `4000)` limits the prompt size.
4. Prompts for `name`, `skills[]`, `projects[{name,` `description}]`, `experience_level` (`fresher|junior|mid`), `education`, `summary`.
5. Calls `askAi` with `temperature:` `0.1` (low, for deterministic extraction).
6. Returns the parsed object directly.

`summarizeResume` simply calls `parseResume` same schema, same logic.

If the user provides no resume, the page builds a default profile locally (`skills:` `["JavaScript",` `"React",` `"Node.js",` `"SQL",` `"Communication"]`) so the exam can still run.

3. ATS checking

`ResumeTips.jsx` posts `{` `resume_text,` `target_role` `}` to `/api/resume/ats-check`. The controller cleans and truncates the text, then asks for:

```
{
  "ats_score": 65,
  "matched_keywords": ["..."],
  "missing_keywords": ["..."],
  "suggestions": ["...", "...", "..."],
  "verdict": "Good Match"
}
```

It uses `temperature:` `0.2` and returns the object unchanged.

The score is **AI-generated from the prompt**. It is not a score from a real applicant tracking system, and two runs on the same resume may differ slightly. Say this clearly in review; it is the honest and technically correct framing.

4. Personalized career guidance

`Guidance.jsx` posts `{` `resume_text,` `target_role` `}` to `/api/resume/guidance`. The controller asks for `candidateName`, `targetRole`, `summary`, `skillsChecklist[{skill,` `status,` `tips}]`, `projectGuidance[{projectName,` `keySellingPoints,` `likelyQuestions[]}]`, `customTechnicalTips[{title,` `content}]` and `customHrTips[{title,` `content}]`, with `temperature:` `0.3`.

The page stores the response in `aiReport`, shows a toast on success, and reads `err.response?.data?.detail` on failure matching the `detail` key this controller uses.

5. Comparison table

Feature	Input	Endpoint	Result
Interview resume analysis	PDF file (FormData, field `resume`)	`POST /api/interview/resume`	`resumeText`, role, experience, projects, skills
Placement resume parsing	Resume text + target role (JSON)	`POST /api/resume/parse`	Structured profile with name, skills, projects, education, summary
ATS check	Resume text + target role	`POST /api/resume/ats-check`	ATS-style score, keyword gaps, suggestions, verdict
Personalized guidance	Resume text + target role	`POST /api/resume/guidance`	Skills checklist, project talking points, technical and HR tips

The lesson: **a resume has no single processing path.** The frontend chooses an endpoint per feature, and each controller defines its own input and output contract.

> **SAY THIS IN REVIEW**
>
> Say this in review "Only the interview upload sends a file; Multer stores it in `server/public` with a 5 MB limit and the controller deletes it after parsing with `pdfjs-dist`. The other three flows send already-extracted text and differ only in prompt and output schema."

TASK 27

Placement test workflow

The placement drive is the largest feature: five test categories, several evaluation calls, a final report and persistence.

1. The five test categories

#	Test	key	Questions	Time	Nature
1	Technical MCQs	`mcq_technical`	12	25 min	Objective, correct answer known
2	Written Answers	`written`	4	30 min	AI-graded long answers with key points
3	Aptitude	`aptitude`	10	20 min	Objective reasoning
4	Soft Skills	`soft_skills`	5	20 min	Scenario answers, AI-graded
5	Project Deep Dive	`project`	4	15 min	AI-graded, auto-skipped with no projects

`PlacementTestPage.jsx` keeps UI state for `step`, `form` (name, role, level, mode), `resumeText`, `resumeData`, `activeTestId`, `questions`, `currentQ`, `selectedOption`, `writtenAnswer`, `testAnswers`, `testScores`, `allStrengths`, `allWeaknesses`, `mcqDetails`, timers and loading messages. It also has helpers:

- `normalizeOptions(options)` accepts strings like `"A)"` `text"`, objects `{label,` `text}`, or a map, and always returns `[{label,` `text}]`.
- `isCorrectAnswer(opt,` `question)` compares by label (case-insensitive) or by option text.

These helpers exist because the AI does not always return the same option shape. That is a practical example of defensive frontend coding against AI output.

2. Step 1 Parse the resume, when provided

If `resumeText` is non-empty, `handleStart` posts to `/api/resume/parse`. Otherwise a default profile is built locally. The result is stored in `resumeData` and the page moves to the summary step.

3. Step 2 Start the test

```
const startRes = await axios.post(ServerUrl + "/api/placement/start", {
  candidateName: form.name, targetRole: form.role, level: form.level, difficultyMode: form.mode
}, { withCredentials: true })
setActiveTestId(startRes.data.testId)
```

The backend checks the user (`404`), credits (`400`), required fields (`400`), creates the document with `status:` `"Incompleted"`, deducts 50 credits and returns `201` `{` `message,` `creditsLeft,` `testId` `}`.

The frontend immediately updates Redux credits from `creditsLeft`. `activeTestId` is the anchor: every later save must reference it.

4. Step 3 Generate the questions

```
await axios.post(ServerUrl + "/api/questions/generate", {
  resume_data: resumeData,
  target_role: form.role,
  test_type: test.key,
  difficulty_mode: form.mode,
  question_count: questionCounts[testNum],
}, { withCredentials: true })
```

`generateQuestionsMaster` dispatches on `test_type`:

test_type	Helper	Prompt output shape
`mcq_technical`	`getTechnicalMCQs`	`{id,` `question,` `options[{label,text}],` `correct_answer,` `explanation}`
`written`	`getWrittenQuestions`	`{id,` `question,` `key_points[],` `difficulty,` `expected_length}`
`aptitude`	`getAptitudeQuestions`	Same as MCQ plus `type` (logical/quantitative/verbal)
`soft_skills`	`getSoftSkillQuestions`	`{id,` `skill_tested,` `scenario,` `question,` `evaluation_criteria[]}`
`project`	`getProjectQuestions`	`{id,` `related_project,` `question,` `key_points[],` `difficulty}`

`getProjectQuestions` returns `[]` when the resume has no projects, and the master controller returns the empty array; the page then skips Test 5 and jumps to the final report.

The response also includes `time_limit_seconds`, scaled from a per-test default:

```
const calculatedTime = Math.round((defaultTime / defaultCount) * count)
```

So the timer adapts if the question count changes.

The router also exposes `/technical`, `/written`, `/aptitude`, `/softskills`, `/project` as separate endpoints. The main frontend flow uses only the master `/generate`, but the individual routes exist for finer control.

5. Step 4 Evaluate responses

Evaluation endpoints and their inputs:

Endpoint	Frontend sends	AI returns
`POST /api/evaluate/mcq`	Array of `{question_id, selected_answer, correct_answer, explanation}`	Deterministic result from `scorer.service.js` no AI call
`POST /api/evaluate/written`	`question`, `answer`, `key_points`, `role`, `level`	`score`, `verdict`, `strengths[]`, `weaknesses[]`, `suggestion`, `missed_concepts[]`, `ideal_answer_hint`
`POST /api/evaluate/softskill`	`scenario`, `question`, `answer`, `role`	`score`, `verdict`, `strengths[]`, `weaknesses[]`, `star_method_used`, `communication_rating`
`POST /api/evaluate/project`	`question`, `answer`, `role`	`score`, `verdict`, `strengths[]`, `weaknesses[]`, `depth_of_knowledge`
`POST /api/evaluate/test-score`	`test_type`, `answers`, `total_questions`	Deterministic score object
`POST /api/evaluate/overall-score`	`test_scores`	Weighted overall score and rating

The page uses written, soft-skill and project evaluation, and pushes `feedback.score` into `testAnswers`. Strengths and weaknesses are accumulated in `allStrengths` / `allWeaknesses` after normalising string-or-array values.

MCQ scoring is local and deterministic. `handleMCQConfirm` compares the selected option with `correct_answer` using `isCorrectAnswer`, pushes a boolean into `testAnswers`, and stores rich details (question, options, selected, correct, explanation) into `mcqDetails`.

> **NOTE**
> Generating and evaluating are separate operations One asks the AI to create assessment content; the other evaluates a candidate's response. Keeping them separate means each can fail and be retried independently, and the evaluation prompts stay short.

6. Step 5 Generate the final report

```
await axios.post(ServerUrl + "/api/evaluate/final-report", {
  candidate_name, target_role, level, test_scores,
  all_weaknesses: [...new Set(allWeaknesses)],
  all_strengths: [...new Set(allStrengths)],
}, { withCredentials: true })
```

`getFinalReport` formats the scores as `- Technical MCQs: 8.3/10 (83%)`, trims strengths to 8 and weaknesses to 10, and asks the AI for `overall_score`, `overall_percentage`, `rating`, `summary`, `positive_areas[]`, `negative_areas[]`, `study_plan[{topic, priority, resource}]`, `readiness_score` and `estimated_ready_in`, at `temperature: 0.35`.

Because the report is generated from submitted test data, **its quality depends on the quality of the preceding questions and evaluations**.

`scorer.service.js` shows the intended deterministic alternative:

Test	Weight
`mcq_technical`	30%
`written`	25%
`aptitude`	20%
`soft_skills`	15%
`project`	10%

`calculateOverallScore` renormalises the weights when a test is missing (so skipping Project does not cap the score at 0.9) and assigns a rating: `Elite` 8.5, `Strong` 7.0, `Developing` 5.5, otherwise `Needs Work`.

> **FINDING**
> The frontend computes scores its own way `handleTestComplete` in the page calculates `score_out_of_10` itself and sends that array to `final-report`. The `/evaluate/test-score` and `/evaluate/overall-score` endpoints, plus the whole weighting service, are available but **not called** by this flow. The AI therefore produces the official rating instead of the deterministic scorer. Both are defensible, but a reviewer may ask which one is authoritative. The answer: today, the AI value that is saved in `finalReport`.

7. Step 6 Save the test attempt

```
await axios.post(ServerUrl + "/api/placement/save", {
  testId: activeTestId, candidateName, targetRole, level, difficultyMode: form.mode,
  resumeText, resumeData, testScores: scores,
  allStrengths, allWeaknesses, finalReport: report,
}, { withCredentials: true })
```

`savePlacementTest` updates the existing document when `testId` resolves, sets `status: "completed"`, and returns `creditsLeft` and `testId`. The page then:

- stores `mcqDetails` in `localStorage` under `placement_mcq_details_<testId>` (so the detailed question review survives a refresh),
- navigates to `/placement-results/<testId>`.

`localStorage` is a pragmatic choice: MCQ details are display-only and were never sent to the backend. The trade-off is that they live only in that one browser.

8. Step 7 View the saved result

`PlacementResultsPage.jsx` reads `id` with `useParams()` and calls:

```
await axios.get(`${ServerUrl}/api/placement/report/${id}`, { withCredentials: true })
```

For `/placement-results/665f...`, `id` is `665f...`. The backend loads the document, checks ownership (`403` if `test.userId.toString()` `!==` `req.userId`), and returns the whole document. The page then renders tabs: Overview, Test Details, Strengths, Study Plan and (if `localStorage` has them) Questions Review.

9. Complete lifecycle

1. **Setup** candidate details + optional resume
2. **Start** POST /placement/start testId, 50 credits
3. **Generate** POST /questions/generate 5 test types
4. **Evaluate** MCQ locally written / softskill / project via AI
5. **Report** POST /evaluate/final-report
6. **Save** POST /placement/save status completed
7. **Retrieve** GET /placement/report/:id results page

testId from step 2 is the handle every later step needs; save and retrieve both use it.

Watch: a missing or stale testId in step 6 silently creates a second drive and charges 50 more credits.

1. **Setup** candidate details and optional resume.
2. **Start** create the record, deduct credits, keep `testId`.
3. **Generate** questions for each category, one call per test.
4. **Evaluate** MCQ locally, written/soft/project via AI.
5. **Report** AI final assessment from the collected scores.
6. **Save** update the drive document with scores, strengths, weaknesses and report.
7. **Retrieve** load the report by id on the results page.

> **SAY THIS IN REVIEW**
>
> Say this in review "`activeTestId` ties the whole drive together: it is created at `/placement/start`, and every later request that persists data references it. Questions are generated per test type through one master endpoint, answers are evaluated per type, and the report is generated, saved and later read by id."

TASK 28

History and reports

This task explains how WeWithU **retrieves** previous attempts instead of generating new ones.

1. History page

`InterviewHistory.jsx` fetches two lists in parallel-ish fashion (two sequential `try/catch` blocks):

Request	Purpose
`GET` `/api/interview/get-interview`	Mock interview history
`GET` `/api/placement/my-tests`	Placement drive history

Both use `withCredentials:` `true`. Results go into local component state:

```
const [interviews, setInterviews] = useState([])
const [placements, setPlacements] = useState([])
```

The page has two tabs, "Mock Interviews (n)" and placement drives, and renders an empty-state message when a list is empty. The `?tab=placements` query parameter preselects a tab.

Backend reads:

```
// getMyInterviews
Interview.find({ userId: req.userId })
  .sort({ createdAt: -1 })
  .select("role experience mode finalScore status createdAt")

// getMyPlacementTests
PlacementTest.find({ userId: req.userId })
  .sort({ createdAt: -1 })
  .select("targetRole level finalReport createdAt status")
```

Both are **read-only**: no AI call, no regeneration. That is the correct design history should be cheap and deterministic.

2. How clicking an interview opens its report

```
onClick={() => navigate(`/report/${item._id}`)}
```

`item._id` is the MongoDB document id. React Router matches `/report/:id`, `InterviewReport.jsx` reads it with `useParams()`, and then:

```
const result = await axios.get(ServerUrl + "/api/interview/report/" + id, { withCredentials: true })
setReport(result.data)
```

`getInterviewReport` loads the interview, returns `404` if missing, recalculates the three averages from the embedded questions, and returns:

```
{ "finalScore": 8, "confidence": 8.1, "communication": 7.4, "correctness": 8.6, "questionWiseScore": [ ... ] }
```

`InterviewReport` passes that object to `Step3Report`, which renders circular progress bars, a Recharts area chart of per-question scores, and a jsPDF export.

3. Placement reports work the same way

`navigate( /placement-results/${item._id} )` `/placement-results/:id` `GET` `/api/placement/report/:id` the full placement document tabs with the saved report.

The difference is the authorization check: the placement report verifies ownership and returns `403`, while the interview report does not.

> **FINDING**
>
> Report id uses two different display names The placement model field is `finalReport` (camelCase) and the API returns it untouched, while the AI prompt uses snake_case keys such as `overall_score` **inside** that object. The page reads `testData.finalReport.overall_score`. This mixed convention is easy to get wrong; a small mapping layer or a consistent convention would help.

4. Where does a report come from?

Three distinct stages:

1. **Evaluation** the AI or deterministic scorer computes scores.
2. **Persistence** the controller saves them in the interview or placement document.
3. **Retrieval** the report endpoint fetches saved data for display.

History and report pages only do stage 3. `Step3Report` also receives the object directly after a fresh interview (`step` `===` `3` in `InterviewPage`), so the same UI works for "just finished" and "loaded later".

> **NOTE**
>
> Useful debugging technique When a report looks wrong, check in this order: (1) is the URL id correct? (2) what does the Axios request send? (3) does the router match? (4) does the controller query return a document? (5) what is the response body shape? Only then suspect the rendering. This separates "no record" from "wrong id" from "frontend bug".

5. Loading versus error states

Page	Behaviour
`InterviewReport.jsx`	Shows "Loading Report..." whenever `report` is `null`. The `catch` only logs. **A failed request shows the loading text forever.**
`PlacementResultsPage.jsx`	Uses `loading` with a `finally` block, plus a separate "No Results Found" screen with a call-to-action button.

That comparison is a good teaching moment:

Loading request in progress.
Success data received.
Error or missing record show a clear message and, where useful, a retry.

The interview report page is missing the third state, which is a one-line fix: add an `error` state in `catch` and render it when set.

> **SAY THIS IN REVIEW**
>
> Say this in review "The history page only reads. Each list item uses its `_id` to build a report URL, the report page reads the id from the route, and the controller queries by that id. Placement also checks ownership, the interview report does not, and the interview report page never leaves its loading state on failure."

TASK 29

Guidance and preparation pages

The question for every page: **does it render local content, fetch from an API, or both?**

1. Guidance page API-driven

`Guidance.jsx` calls:

```
await axios.post(ServerUrl + "/api/resume/guidance",
  { resume_text: resumeText, target_role: targetRole },
  { withCredentials: true })
```

It stores the response in `aiReport`, shows loading state while the request runs, and toasts success or `err.response?.data?.detail`. Resume text comes from a PDF parsed in the browser or from manual typing. The page also keeps **static** content: technical, HR, group-discussion and coding tips plus a checklist whose progress is tracked in `checkedItems`.

Verdict: mixed page static tips plus an API-driven personalized report.

2. Resume Tips page API-driven ATS analysis

`ResumeTips.jsx` extracts text with browser PDF.js, then:

```
await axios.post(ServerUrl + "/api/resume/ats-check",
  { resume_text: resumeText, target_role: targetRole }, { withCredentials: true })
```

It renders `ats_score`, `matched_keywords`, `missing_keywords`, `suggestions` and `verdict`, with loading state and toasts. The analysis is produced by the backend, not hardcoded.

Verdict: API-driven, with static explanatory sections around it.

3. Company Prep page local data only

To classify it correctly, inspect imports and network calls. `CompanyPrep.jsx` imports only React, the router, icons, `Navbar`, `Footer` and `motion`. There is **no axios import and no `fetch` call**. Its `useState` values are `selectedCategory`, `selectedCompany`, `activeTab`, `filterDifficulty`, `searchQuery` and an image fallback counter all UI state. Company details live in local arrays and JSX; images have a fallback chain.

Verdict: frontend-only. Everything renders instantly, and nothing is stored per user.

4. Roadmap page local data

`Roadmap.jsx` imports React, the router, icons and layout components again no axios and no fetch. `FIELDS` and `ROADMAPS` are local constants; `pathData = ROADMAPS[selectedField] || DEFAULT_ROADMAP`. The page mentions Redux and TanStack Query **as topics to learn**, which does not mean the page uses them to fetch data.

Verdict: frontend-only.

> **WATCH OUT**
>
> Do not infer architecture from content A page is not "AI-powered" because it talks about interviews, and not "API-driven" because it mentions Redux. Classification comes from imports, `axios` / `fetch` calls, and where the displayed data comes from.

5. Feature classification summary

Page	Main behaviour, established from source
`Home.jsx`	Static marketing plus links; reads user from Redux
`Auth.jsx`	Firebase popup `POST /api/auth/google` Redux
`InterviewPage.jsx`	Hosts setup session report steps
`Step1SetUp.jsx`	`POST /api/interview/resume`, `POST /api/interview/generate-questions`
`Step2Interview.jsx`	`POST /api/interview/submit-answer`, `POST /api/interview/finish`
`Step3Report.jsx`	Pure rendering + client-side PDF export
`InterviewHistory.jsx`	Two GET requests, tabs
`InterviewReport.jsx`	`GET /api/interview/report/:id`
`Pricing.jsx`	`POST /api/payment/order`, Razorpay checkout, `POST /api/payment/verify`
`PlacementTestPage.jsx`	parse, start, generate, evaluate 3, final-report, save
`PlacementResultsPage.jsx`	`GET /api/placement/report/:id` + `localStorage` details
`Guidance.jsx`	`POST /api/resume/guidance` + static tips
`ResumeTips.jsx`	`POST /api/resume/ats-check` + static tips
`CompanyPrep.jsx`	Frontend-only data
`Roadmap.jsx`	Frontend-only data
`Navbar.jsx`	`GET /api/auth/logout`, Redux user, theme in `localStorage`
`Footer.jsx`, `Logo.jsx`, `Timer.jsx`, `AuthModel.jsx`	Presentation only

The takeaway: **static and API-driven content are different architectural choices.** Static content renders immediately with no request and no failure mode; dynamic content needs an endpoint, a loading state and an error state.

> **SAY THIS IN REVIEW**
>
> Say this in review "Guidance and Resume Tips call the AI through the resume router. Company Prep and Roadmap are pure frontend arrays with no network calls. I decided that by reading the imports and looking for axios calls, not by reading the page titles."

V

Review Pack

WEWITHU REVIEW NOTES

Findings register, end-to-end flow gallery, answer keys, rapid-fire viva questions and a one-page cheat sheet.

This part is what you open the night before, or during, the review.

A. Consolidated findings register

Every observation below comes from the code in this repository. "Severity" is about impact on correctness, security or marks.

#	Finding	File / line area	Severity
1	Default port `6000` while the frontend defaults to `8000` and `.env.example` says `8000`	`server/index.js`, `client/src/utils/serverUrl.js`	High (broken dev setup without `.env`)
2	`connectDb()` runs inside the `listen` callback; server accepts traffic before the DB is ready	`server/index.js`	High
3	DB connection failure is only logged; process continues without a database	`server/config/connectDb.js`	Medium
4	Missing token returns `400`; invalid/expired token returns `500`	`server/middlewares/isAuth.js`	High (semantics)
5	`if (!verifyToken)` branch is effectively unreachable	`isAuth.js`	Low
6	`savePlacementTest` updates a test by id without an ownership check	`placement.controller.js`	**Critical (IDOR)**
7	`getInterviewReport` returns any interview by id with no ownership check	`interview.controller.js`	**Critical (IDOR)**
8	`submitAnswer` has no null interview check, no index bounds check, no ownership check	`interview.controller.js`	High
9	`createOrder` trusts client-supplied `amount` and `credits`	`payment.controller.js`	**Critical (price tampering)**
10	Duplicate email surfaces as `500` instead of `409`	`auth.controller.js`	Medium
11	`res.status(200).json(user)` returns the whole document	`user.controller.js`, `payment.controller.js`	Medium
12	Cookie uses `secure: false` (and `sameSite: "lax"`)	`auth.controller.js`	Medium in production
13	AI evaluation values are saved without validation or clamping	`interview.controller.js`	Medium
14	Error objects interpolated into client-visible messages	many controllers	Medium (info leak)
15	Inconsistent error key (`message` vs `detail`) and response shapes	all controllers	Low
16	No global 404 handler and no Express error middleware	`server/index.js`	Low
17	`logout` is a `GET` request	`auth.route.js`, `Navbar.jsx`	Low
18	Multer has no MIME/extension filter; filename uses client-provided text	`middlewares/multer.js`	Medium
19	`placement.save` fallback silently creates a second drive and charges 50 more credits	`placement.controller.js`	High
20	Create + credit deduction are two writes with no transaction	`startPlacementTest`	Medium
21	Schema default `status: "completed"` contradicts `startPlacementTest`'s explicit `Incompleted`	`placement.model.js`	Low
22	`InterviewReport.jsx` never exits the loading state on error	`client/src/pages/InterviewReport.jsx`	Medium (UX)
23	`/evaluate/test-score`, `/evaluate/overall-score` and `scorer.service.js` are unused by the main flow	frontend / server	Low
24	Browser PDF.js is loaded from a CDN at runtime	`Guidance.jsx`, `ResumeTips.jsx`, `PlacementTestPage.jsx`	Low (offline/prod dependency)
25	Firebase web config has hardcoded fallbacks in source	`client/src/utils/firebase.js`	Low (public by design, still noisy)

The three fixes to mention first (best value for time)

1. **Ownership checks** in `savePlacementTest` and `getInterviewReport`, copied from the pattern already used in

`getPlacementTestReport`:

```
if (test.userId.toString() !== req.userId) {
  return res.status(403).json({ message: "Unauthorized access to this report." });
}
```

2. **Status codes in `isAuth`**: return `401` for missing and for invalid/expired tokens, and log the verification error instead of returning it.
3. **Server-side pricing**: ignore `amount` and `credits` from the body and derive them from a plan table keyed by `planId`.

B. End-to-end flows you should be able to draw

Flow 1 Login and session

```
Auth.jsx
  signInWithPopup(auth, provider)          Firebase Google popup
  POST /api/auth/google { name, email }    withCredentials
    auth.controller.googleAuth
      User.findOne({ email })      create if missing (credits: 100)
      genToken(user._id)           jwt.sign({ userId }, JWT_SECRET, 7d)
      res.cookie("token", ...)     HttpOnly
      return 200 + user document
  dispatch(setUserData(user))
```

Flow 2 Current user (Task 18)

```
App.jsx  GET /api/user/current-user (withCredentials)
  index.js  cors   express.json   cookieParser   /api/user
  user.route.js  GET /current-user   isAuth   getCurrentUser
  isAuth  req.cookies.token   jwt.verify   req.userId   next()
  getCurrentUser  User.findById(req.userId)
  200 JSON     dispatch(setUserData(result.data))
```

Flow 3 Interview question generation

```
Step1SetUp  POST /api/interview/generate-questions { role, experience, mode, resumeText, projects, skills, numQuestions,
  difficulty }
  generateQuestion
    validate role/experience/mode        400
    User.findById(req.userId)            404
    credits < 50                         400
    askAi(messages, {json, returnParsed})
    validate questions array             500
    user.credits -= 50 ; user.save()
    Interview.create({ ..., questions: [{question, difficulty, timeLimit}] })
  200 { interviewId, creditsLeft, userName, questions }
Redux interview.interviewData      navigate("/interview-session")
```

Flow 4 Answer submission and finish

```
Step2Interview  POST /api/interview/submit-answer { interviewId, questionIndex, answer, timeTaken }
  submitAnswer
    interview = Interview.findById(interviewId)
    question  = interview.questions[questionIndex]
    no answer / time exceeded   score 0 + feedback, save, return
    askAi(evaluation prompt)
    question.{answer, confidence, communication, correctness, score, feedback}
    interview.save()
  200 { feedback }

POST /api/interview/finish { interviewId }
  finishInterview   averages   finalScore + status "completed"   save
  200 { finalScore, confidence, communication, correctness, questionWiseScore }
```

Flow 5 Placement drive (Task 27)

```
POST /api/resume/parse            (optional) resume_text   structured profile
POST /api/placement/start           201 { testId, creditsLeft }  (50 credits)
POST /api/questions/generate       5  (one per test type, time_limit_seconds)
MCQ answers                       scored locally in the page
POST /api/evaluate/written         4
POST /api/evaluate/softskill       5
POST /api/evaluate/project         4   (skipped if no projects)
POST /api/evaluate/final-report     AI report
POST /api/placement/save            update document, status "completed"
navigate(`/placement-results/${testId}`)
GET  /api/placement/report/:id      ownership check   document   tabs
```

Flow 6 Payment and credits

```
Pricing.jsx  POST /api/payment/order { planId, amount, credits }
  createOrder   razorpay.orders.create({ amount: amount*100, currency: "INR" })
                Payment.create({ userId, planId, amount, credits, razorpayOrderId, status: "created" })
  200 order      Razorpay checkout opens
handler(response)  POST /api/payment/verify { razorpay_order_id, razorpay_payment_id, razorpay_signature }
  verifyPayment
    HMAC-SHA256(order_id + "|" + payment_id, RAZORPAY_KEY_SECRET) === signature?    else 400
    Payment.findOne({ razorpayOrderId })   404 if missing
```

```
    status already "paid"   200 { message: "Already processed" }
    payment.status = "paid" ; payment.razorpayPaymentId = ... ; save()
    User.findByIdAndUpdate(userId, { $inc: { credits } }, { new: true })
200 { success, message, user }     dispatch(setUserData(user))
```

C. Answer keys for the practice questions

Phase 4

1. **What does `app.use('/api/user', userRouter)` do? It mounts the user router under the `/api/user` prefix.** (It does not connect to MongoDB and does not verify a JWT.)
2. **Why does `isAuth.js` set `req.userId`? To let later handlers access the verified token's user ID** without re-parsing or re-verifying.
3. **Valid token but `User.findById()` finds no user? The controller returns 404.** Authentication succeeded; the record is missing.
4. **Conventional status for missing or invalid credentials? 401.**
5. **Which file defines the current-user route's middleware and controller sequence? `server/routes/user.route.js`.** (`store.js` is Redux; `user.model.js` is the schema.)

Phase 5

1. **Which environment variable does `connectDb.js` use? `MONGODB_URL`.**
2. **What does `timestamps: true` add? `createdAt` and `updatedAt`.**
3. **Which operation creates a new placement-test document? `PlacementTest.create(...)`.**
4. **Does `unique: true` mean duplicate emails always produce a normal Mongoose validation error? No. Duplicates produce a MongoDB duplicate-key error (code 11000).**
5. **Why must the backend validate input even when the frontend does? Frontend checks can be bypassed**; the API can be called directly.

Phase 6

1. **Which function sends the interview prompts to the AI service? `askAi()`.**
2. **What does the interview resume endpoint receive? A PDF uploaded in the `resume` field** (`multipart/form-data`, handled by `upload.single("resume")`).
3. **Why keep `activeTestId`? To identify the test attempt across later requests** (save, and building the results URL).
4. **How does clicking an interview in history open its report? It navigates to `/report/:id` using the interview document id**, which the report page sends to `GET /api/interview/report/:id`.
5. **Is a page AI-powered just because it has interview-prep content? No; inspect the actual data source and API calls.** (`CompanyPrep` and `Roadmap` are static.)

D. Rapid-fire viva questions with model answers

1. **Why is `dotenv.config()` called before anything else?**

To load `.env` into `process.env` so `PORT`, `MONGODB_URL`, `JWT_SECRET`, `GROQ_API_KEY` and Razorpay keys are available. Because ESM hoists imports, modules that read env at import time (the Razorpay service) call `dotenv.config()` themselves.

2. **Does `const app = express()` start a server?**

No. It builds the request pipeline. `app.listen()` starts the HTTP server.

3. **What exactly does `credentials: true` do in CORS?**

It tells the browser that credentialed cross-origin requests (cookies, auth headers) are allowed, and paired with `withCredentials: true` on Axios the cookie is actually sent and stored.

4. **If the cookie is not sent, where do you look first?**

`withCredentials` on the Axios call, `credentials: true` in CORS, the origin being in the allowlist, and the cookie's `SameSite` / `Secure` attributes.

5. **What happens if `express.json()` is missing?**

`req.body` is `undefined` for JSON requests, and controllers throw on `req.body.role`. Multer routes would still work because they parse multipart themselves.

6. **Why is the prefix plus path combination important in debugging?**

Because the endpoint you quote must be the concatenation; searching only the router for a full URL, or only `index.js` for a handler, will not find it.

7. **Explain `next()` in one sentence.**

"Continue to the next handler in the pipeline"; without it, and without sending a response, the request hangs.

8. **What is the difference between `401` and `403` here?**

`401` means I cannot verify you (no/expired token). `403` means I know you, but this record belongs to someone else exactly what `getPlacementTestReport` returns.

9. **Why does `getCurrentUser` not verify the token again?**

`isAuth` already did it and placed `req.userId` on the request. Re-verifying would duplicate work and blur responsibilities.

10. **Where does the JWT payload come from, and what does it contain?**

`server/config/token.js` signs `{ userId }` with `JWT_SECRET` for 7 days. `isAuth` reads `verifyToken.userId`.

11. **What is `unique: true` really?**

An index constraint, not a validator. Duplicates raise a MongoDB duplicate-key error.

12. **Why embed questions inside the interview instead of a separate collection?**

Questions belong to one interview and are always read with it; embedding gives a single read and avoids joins. Denormalisation is

safe here because questions are never shared.

13. **Why is `.select(...)` used in the history queries?**
To return only the fields the list needs, reducing payload size and avoiding leaking the full document.
14. **What does `$inc` give you in `verifyPayment`?**
An atomic increment performed by MongoDB, so concurrent verifications cannot lose an update.
15. **Why does the AI not write to the database?**
It only returns content. The controller validates the response and decides what to persist, keeping AI failures away from the data layer.
16. **What is the difference between generating questions and evaluating answers?**
Generation creates assessment content; evaluation scores the candidate's response. They are separate endpoints with separate prompts and separate failure modes.
17. **How would you fix the "Loading Report..." forever bug?**
Add an `error` state set in `catch`, and render an error panel with a retry or a back link. `PlacementResultsPage` already demonstrates the pattern with `loading` in `finally` and a "No Results Found" screen.
18. **How do you know `CompanyPrep` is frontend-only?**
It imports no `axios` and calls no `fetch`; all data are local constants and all `useState` values are UI state.
19. **Which single addition would most improve this backend's security?**
A shared ownership-check helper applied to every by-id read and update, plus server-side plan pricing.
20. **If a request returns 500 for an expired token, what breaks in the frontend?**
The UI cannot distinguish "log in again" from "server broken", so it does not redirect to `/auth`, and monitoring counts a normal auth event as a server error.

E. Two-day revision plan

Slot	Focus	What to do
Day 1 1 h	Phases 4	Read Tasks 15 19; open `index.js`, `user.route.js`, `isAuth.js` and say the trace out loud
Day 1 1 h	Phase 5	Read Tasks 20 23; draw the four schemas from memory, then compare
Day 2 1 h	Phase 6	Read Tasks 24 29; rehearse the interview and placement flows on the diagram
Day 2 30 min	Review pack	Memorise the three top fixes and section D answers 1 10
Day 2 30 min	Demo practice	Run the app, open DevTools Network, click through login interview history

F. Demo script for the review (10 minutes)

1. **Show the run command and the port.** Point out that `PORT` must be `8000` to match the frontend, or `VITE_API_URL` must match the backend.
2. **Open DevTools Network** and log in. Show `POST` `/api/auth/google` returning `200` with a `Set-Cookie` for `token`.
3. **Reload the page.** Show `GET` `/api/user/current-user` with the cookie in Request Headers, and `200` with the user document. This is Task 18 in one screenshot.
4. **Start an interview.** Show `POST` `/api/interview/generate-questions` and the response containing `interviewId` and `questions`; show credits drop by 50 in the navbar.
5. **Answer one question.** Show the AI feedback returned in `{ feedback }`.
6. **Finish and open history.** Show the report request by id.
7. **Run a placement drive** if time allows: start generate save results.
8. **Then say one finding and its fix** (ownership check or `401` in `isAuth`). Ending with an improvement you identified reads as engineering maturity, not as criticism of someone else's code.

G. Glossary

Term	Meaning in this project
Bootstrap	Starting the backend: env app middleware routes listen
CORS	Browser rule for cross-origin responses; configured with an origin allowlist and `credentials: true`
JWT	Signed token holding `{ userId }`, signed with `JWT_SECRET`, valid 7 days
HttpOnly cookie	Cookie named `token` that JavaScript cannot read, sent automatically with `withCredentials`
Middleware	Function in the pipeline that can `next()`, respond, or pass an error
Router	`express.Router()` instance mapping method + path to middleware and controller

Term	Meaning in this project
Mount prefix	The first argument of `app.use`, e.g. `/api/interview`
Controller	Function holding business logic; validation, model calls, response
Model	Mongoose wrapper for a collection with a schema
Schema	Definition of fields, types, defaults, enums, required and indexes
Embedded document	A subdocument stored inside the parent, like `questions` in an interview
ObjectId	MongoDB's 12-byte document id, used as `userId` in the JWT
`ref`	Declares a reference to another model; not enforced by the database
Projection / `select`	Returning only chosen fields
Populate	Auto-fetching a referenced document; not used here
Upsert fallback	Create-if-missing path in `savePlacementTest`
Idempotent	Safe to call twice with the same effect; payment verify is partly idempotent
HMAC	Hash-based signature used to verify Razorpay payments
`askAi`	Shared Groq wrapper with JSON mode, extraction and 429 retries
Prompt	The system + user messages sent to the model
JSON mode	`response_format: { type: "json_object" }` asking the model for JSON
Temperature	Randomness control; low (0.1) for parsing, higher (0.35) for reports
Rate limit (429)	Provider throttle; `askAi` waits and retries up to 3 times
IDOR	Insecure Direct Object Reference: reading or updating a record by id without an ownership check
409 Conflict	The correct status for a duplicate resource such as a repeated email
DTO / projection	A shaped response containing only the fields the client needs

H. One-page cheat sheet

```
BOOTSTRAP      dotenv   express()   cors(allowlist, credentials)   express.json()   cookieParser()
                 app.use(prefix, router)  8   app.listen(PORT, () => connectDb())
               PORT default 6000, .env.example 8000, frontend default 8000    mismatch

AUTH           cookie token   jwt.verify(JWT_SECRET)   req.userId   next()
               missing: 400 (should be 401)   invalid: 500 in catch (should be 401)

CURRENT USER   App.jsx   /api/user   user.route   isAuth   getCurrentUser   User.findById   200 JSON   Redux

DATABASE       mongoose.connect(MONGODB_URL); connectDb inside listen; errors only logged

MODELS         User(name, email unique, credits 100)
               Interview(userId ref, role, experience, mode enum, questions[] embedded, finalScore, status)
               Payment(userId ref, planId, amount, credits, razorpayOrderId, razorpayPaymentId, status enum)
               PlacementTest(userId ref, candidateName, targetRole, level, difficultyMode, resumeData,
                             testScores[], allStrengths[], allWeaknesses[], finalReport, status default completed)

DATA LIFECYCLE create   PlacementTest.create + user.save (no transaction)
               read     find({userId}).sort().select()
               update   findById + assign + save (no ownership check)

VALIDATION     frontend (UX)   controller (business + ownership)   schema (type/required/enum/default)
               unique index (email   11000, returned as 500 instead of 409)

INTERVIEW      setup   generate-questions (deduct 50, create doc)   submit-answer (AI scores, save)
                 finish (average, status completed)   history/report by id

AI             askAi(messages, { json, returnParsed, temperature, max_tokens })
               extractJson handles raw, fenced, first-brace region   429 retry  3

RESUME         4 flows: file upload (Multer 5 MB), text parse, ATS check, guidance

PLACEMENT      5 tests: mcq_technical 12/25m   written 4/30m   aptitude 10/20m
               soft_skills 5/20m   project 4/15m (auto-skip)
               start(50 credits)   generate  5   evaluate   final-report   save   report/:id

STATUS CODES   400 bad request   401 unknown identity   403 not allowed   404 missing   409 duplicate   500 server
```